KYRO_ENTERPRISE_DOCUMENTATION

KYRO AML Risk Assessment Platform

Complete Enterprise-Grade Project Documentation

> **Document Classification:** Internal Technical Reference | Enterprise Handover Document
> **Version:** 1.0.0
> **Last Updated:** 2026-07-30
> **Standards:** IEEE SRS 830, UML 2.5, C4 Architecture Model, OWASP Top 10, FATF AML Guidelines

---

Table of Contents

---

1. Project Overview

1.1 Project Name

KYRO — *Know Your Risk, Own the Outcome*

KYRO is an enterprise-grade **Anti-Money Laundering (AML) Risk Assessment Platform** designed for financial institutions, fintech companies, and regulatory compliance teams. It combines a deterministic rules engine (Phase 1) with an ensemble machine learning scoring engine (Phase 2) to detect, score, explain, and alert on suspicious financial transactions in real time.

1.2 Project Objective

Simple explanation: KYRO's goal is to automatically detect transactions that might be used to launder money or finance terrorism, score how risky they are on a scale of 0–100, and alert compliance teams with clear explanations so they can investigate quickly.

Technical objective: Build a two-phase, API-driven risk assessment pipeline that:

1. Ingests transaction data via REST API
2. Runs deterministic rule checks (R001–R010) synchronously at ingestion
3. Applies an ensemble ML model (RF Regressor + RF Classifier + Isolation Forest) to compute a probabilistic risk score
4. Generates SHAP-based natural language explanations
5. Routes high-risk scores to an analyst alert work queue
6. Supports model versioning, A/B testing, and automated retraining

---

1.3 Problem Statement

Business Problem (Simple):
Banks and financial companies are legally required to monitor millions of daily transactions for suspicious activity (money laundering, fraud, terrorist financing). Doing this manually is:

- Too slow (analysts cannot review every transaction)
- Error-prone (human fatigue, missed patterns)
- Expensive (requires large compliance teams)
- Reactive (problems found after the fact)

Regulatory Context:

- **FATF** (Financial Action Task Force) mandates transaction monitoring
- **AML** (Anti-Money Laundering) regulations in 200+ countries require SAR (Suspicious Activity Reports)
- Non-compliance leads to multi-billion dollar fines (e.g., HSBC paid $1.9B in 2012)

Technical Problems:

1. No real-time automated transaction scoring system
2. Rule-based systems generate excessive false positives (70–80% typical)
3. No explainability for ML decisions (regulatory requirement)
4. No model versioning or A/B testing capability
5. No background ETL safety net for bulk-loaded transactions

1.4 Existing System

What most banks use today:

- Manual review by compliance analysts
- Simple threshold-based rules (e.g., "flag all transactions > $10,000" — a US FinCEN CTR rule)
- Spreadsheet-based tracking
- Legacy core banking system alerts with no ML

Drawbacks of the Existing System:

Drawback	Impact
High false positive rate (70–80%)	Analysts waste time on non-suspicious alerts
No behavioral baseline	Cannot detect gradual pattern changes
No ML learning from feedback	Rules never improve
No explainability	Cannot justify alerts to regulators
No real-time scoring	Suspicious transactions might process before detection
Manual rule maintenance	Rules become outdated quickly
No model versioning	Cannot safely update detection algorithms

1.5 Proposed Solution

KYRO implements a **dual-phase AML risk assessment architecture**:

```
Phase 1: Rules Engine (Deterministic)
  ├── R001: Amount Threshold (> $10,000)
  ├── R002: Velocity Daily (> 5 txn/day)
  ├── R003: Velocity Hourly (> 3 txn/hour)
  ├── R004: High Risk Country (FATF list)
  ├── R005: PEP Match
  ├── R006: Sanctions Match (CRITICAL)
  ├── R007: New Counterparty
  ├── R008: Weekend Activity
  ├── R009: Round Amount
  └── R010: Rapid Succession (< 60 seconds apart)

Phase 2: ML Engine (Probabilistic + Explainable)
  ├── Model 1: Random Forest Regressor → Risk Score (0–100)
  ├── Model 2: Random Forest Classifier → Anomaly Probability (0.0–1.0)
  ├── Model 3: Isolation Forest → Unsupervised Outlier Score
  ├── Ensemble Combiner → Weighted Combined Score
  └── SHAP Explainer → Natural Language Explanation
```

1.6 Business Requirements

ID	Requirement
BR-001	System shall score 100% of ingested transactions within 500ms
BR-002	System shall generate compliance-ready alert explanations
BR-003	System shall support FATF-mandated high-risk country detection
BR-004	System shall generate Suspicious Activity Report (SAR) recommendations
BR-005	System shall maintain an immutable audit log of all actions
BR-006	System shall support role-based access (Analyst/Compliance/Admin)
BR-007	System shall retrain ML models weekly with fresh data
BR-008	System shall support A/B testing before model promotion

---

1.7 Functional Requirements

ID	Requirement	Priority
FR-001	User registration and JWT authentication	Must Have
FR-002	Transaction ingestion (single and batch)	Must Have
FR-003	Synchronous rules engine scoring on ingestion	Must Have
FR-004	ML scoring endpoint (POST /ml/score-transaction)	Must Have
FR-005	Batch ML scoring (POST /ml/score-batch)	Must Have
FR-006	Customer-level risk profiling (POST /ml/score-customer)	Should Have
FR-007	KYC review management	Must Have
FR-008	Alert work queue (OPEN/ASSIGNED/RESOLVED/ESCALATED)	Must Have
FR-009	PEP and Sanctions screening	Must Have
FR-010	SHAP explanation for every ML decision	Must Have
FR-011	Model training API (POST /ml/train)	Must Have
FR-012	Model version listing (GET /ml/models)	Should Have
FR-013	Performance monitoring (GET /ml/performance)	Should Have
FR-014	Audit log for all create/update/delete operations	Must Have
FR-015	Background ETL Celery task (daily)	Should Have
FR-016	Automated ML retraining check (weekly)	Should Have
FR-017	Pagination on all list endpoints	Should Have
FR-018	Health check endpoint (GET /api/v1/health)	Must Have

1.8 Non-Functional Requirements

Category	Requirement
Performance	API response time < 500ms for scoring (p95)
Scalability	Horizontal scaling via Docker Compose service replicas
Security	JWT HS256 auth, bcrypt passwords, RBAC with 3 roles
Availability	`restart: unless-stopped` on all critical services
Auditability	Immutable `audit_logs` table; BRIN index on `performed_at`
Explainability	Every ML decision includes top-5 SHAP feature contributions

Category	Requirement
Maintainability	Clean architecture: routers/services/models/schemas separation
Testability	pytest with TestClient + in-memory SQLite fixture
Portability	Fully containerized via Docker Compose

1.9 Project Scope

In Scope:

- Transaction ingestion, storage, and retrieval
- Deterministic rule-based risk scoring (10 rules)
- ML ensemble scoring (3 models)
- SHAP explainability
- Alert management workflow
- KYC review management
- PEP/Sanctions screening
- JWT authentication and RBAC
- Celery background tasks (ETL + ML retraining)
- Model registry with A/B testing
- Audit logging
- Docker Compose deployment

Out of Scope (Future Enhancements):

- Frontend UI/dashboard (no React or Angular in this phase)
- External data feeds (Refinitiv, Dow Jones sanctions lists)
- Real-time streaming (Kafka integration)
- Graph network analysis (customer network patterns)
- GDPR right-to-be-forgotten handling

1.10 Expected Outcome

1. Every transaction receives a risk score (0–100) within 500ms of ingestion
2. Compliance analysts see only high-signal alerts with clear explanations
3. ML models automatically improve via weekly retraining on labeled analyst feedback

4. Full audit trail satisfies regulatory examination requirements
5. False positive rate < 30% (vs. industry average 70–80%) via ML signal

---

1.11 Business Benefits

Benefit	Quantification
Reduced analyst workload	~60% fewer manual reviews via ML pre-filtering
Faster alert triage	< 2 minutes per alert (SHAP explanation pre-generated)
Regulatory compliance	Meets FATF, FinCEN, EU AMLD requirements
Lower false positive cost	Estimated $500K/year analyst time savings per 1M txn/month
Model A/B testing	Zero-downtime model updates
Continuous improvement	Weekly retraining uses analyst feedback loop

---

1.12 Future Enhancements

1. **Graph Analytics** — Detect money mule networks via customer relationship graphs (NetworkX/Neo4j)
2. **Kafka Streaming** — Replace batch ETL with real-time event streaming
3. **LLM Integration** — GPT-4/Claude for narrative Suspicious Activity Report drafting
4. **External Watchlist API** — Refinitiv World-Check, OFAC SDN list live integration
5. **Frontend Dashboard** — React + D3.js risk visualization dashboard
6. **Drift Detection** — Automated model performance degradation alerts
7. **Federated Learning** — Cross-institution learning without data sharing
8. **Mobile App** — Analyst alert review on mobile devices

---

2. High Level Architecture

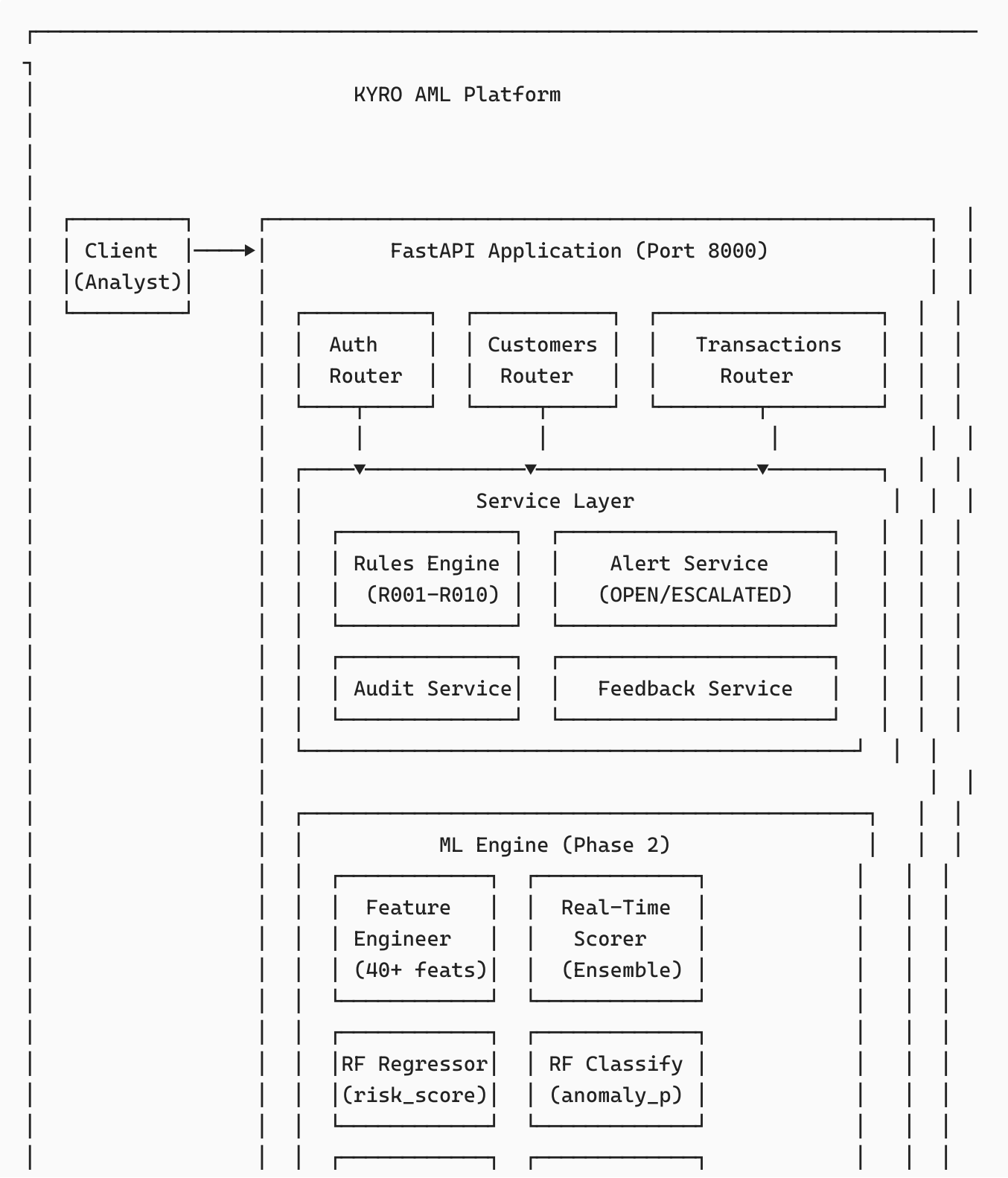
2.1 Architecture Overview (Simple Explanation)

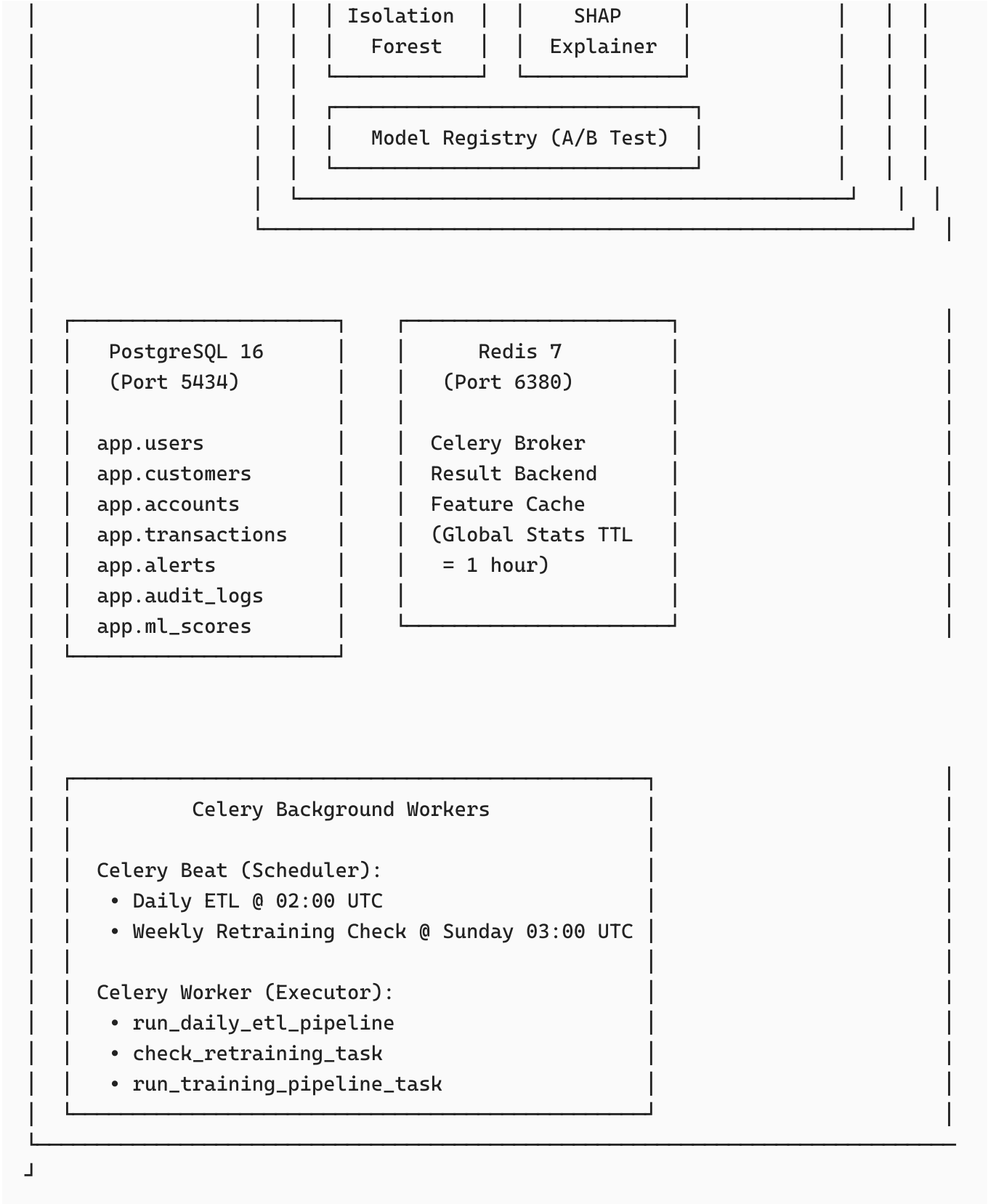
Think of KYRO like an airport security system:

- The **luggage belt** = Transaction API (things come in)
- The **X-ray machine** = Rules Engine (basic automated checks)

- The **AI scanner** = ML Engine (advanced pattern detection)
- The **security officer alert** = Alert Service (flag for human review)
- The **security log** = Audit Service (record everything)
- The **control room** = Redis + Celery (background coordination)
- The **database** = PostgreSQL (permanent storage)

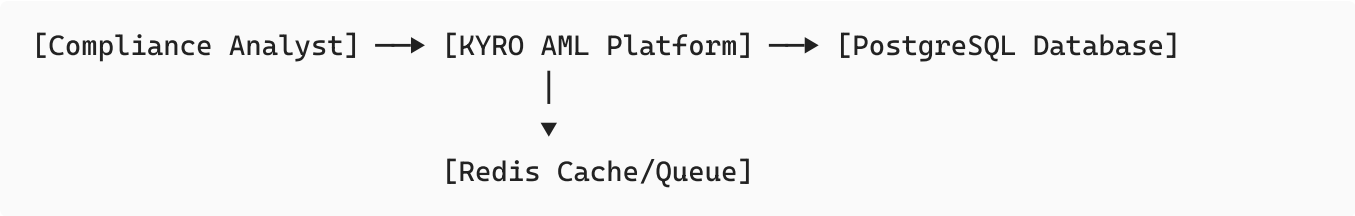
2.2 System Architecture Diagram



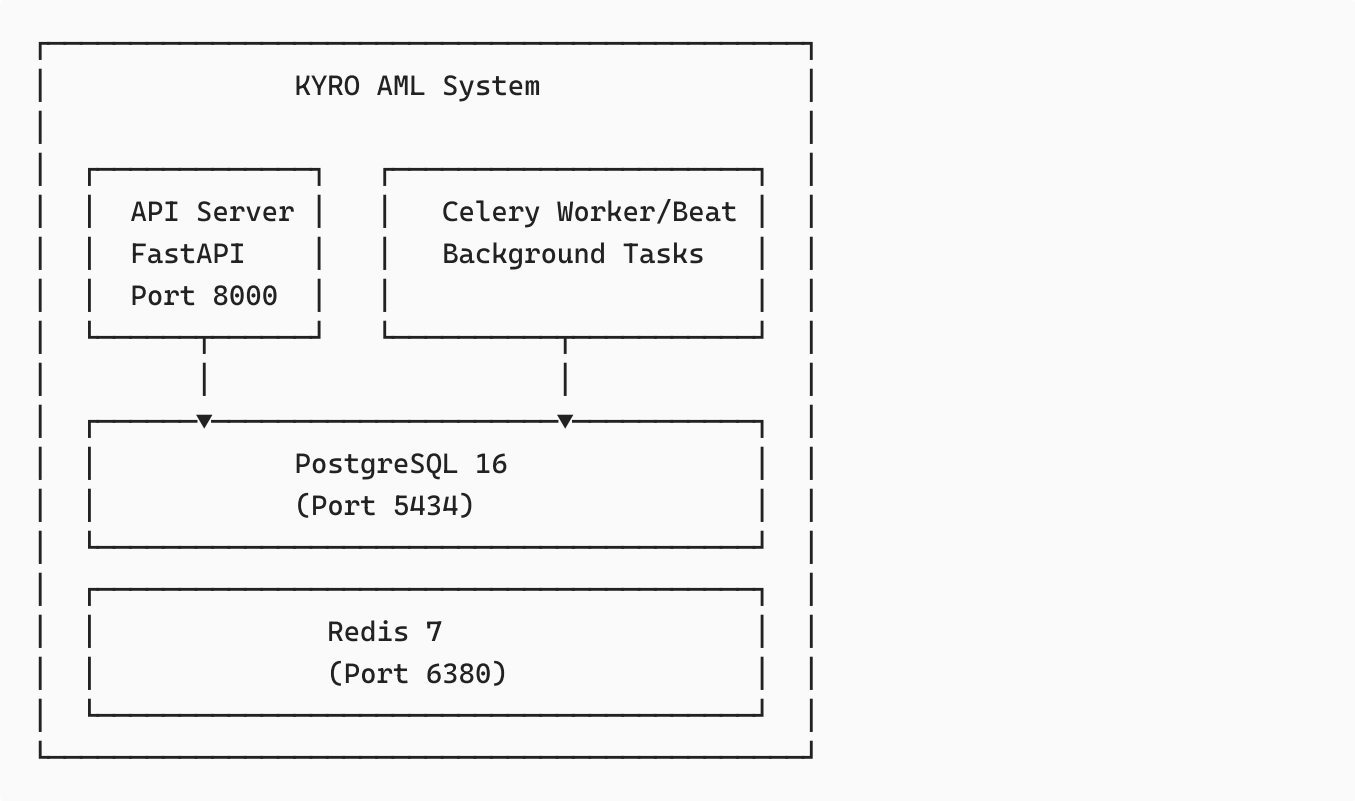


2.3 C4 Architecture Model

Level 1 — System Context:



Level 2 — Container View:



2.4 Component Explanations

Component	Why It Exists	What It Does
FastAPI API Server	High-performance async HTTP framework	Handles all client requests, routing, validation
PostgreSQL 16	Relational ACID-compliant database	Permanent storage for all entities
Redis 7	In-memory data store	Celery broker/backend + feature caching
Celery Worker	Async task execution	Runs ETL and ML training without blocking API
Celery Beat	Cron scheduler	Triggers daily ETL and weekly retraining
Rules Engine	Deterministic AML rules (R001-R010)	Fast, auditable first-pass risk scoring

Component	Why It Exists	What It Does
ML Engine	Probabilistic ensemble scoring	Detects patterns rules cannot
SHAP Explainer	Model explainability	Produces regulatory-compliant explanations
Model Registry	Version management	Safe A/B testing before model promotion
Audit Service	Immutable action logging	Regulatory and forensic trail
Alert Service	Risk routing	Converts scores to analyst work items

2.5 Why This Architecture Was Chosen

Why FastAPI over Flask/Django?

- FastAPI has native async support and built-in OpenAPI/Swagger docs
- Pydantic v2 validation is 10–50x faster than Flask's Marshmallow
- Type annotations enable auto-generated documentation
- Django is too heavyweight for a microservice API

Why PostgreSQL over MongoDB?

- Transactions require ACID guarantees (PostgreSQL)
- JSON fields (JSONB) offer MongoDB-like flexibility for risk_flags
- Advanced indexing (BRIN for time-series, GIN for JSONB)
- Financial data requires relational integrity (foreign keys)

Why Redis over RabbitMQ?

- Redis is already needed for feature caching (dual purpose)
- For the task volume in this system, Redis pub/sub is sufficient
- RabbitMQ adds operational complexity without benefit at this scale

Why sklearn over TensorFlow/PyTorch?

- Random Forest and Isolation Forest are interpretable and fast
- SHAP TreeExplainer works natively with sklearn tree ensembles
- No GPU required, lower infrastructure cost
- Deep learning is overkill for tabular financial data at this scale

---

3. Technology Stack

3.1 Backend Framework

FastAPI (>=0.115.0)

What it is: A modern, high-performance Python web framework for building REST APIs. It is built on top of Starlette (async web framework) and Pydantic (data validation).

Why it is used: KYRO needs fast, well-documented, type-safe APIs with built-in validation. FastAPI provides all of this out of the box.

Why it was selected over alternatives:
Feature	FastAPI	Flask	Django REST
Performance	★★★★★ (async native)	★★★ (sync default)	★★★
Auto Swagger Docs	✅ Built-in	❌ Extension needed	❌ Extension needed
Type Validation	✅ Pydantic native	❌ Manual	✅ Serializers
Learning Curve	Medium	Low	High
Version Used	>=0.115.0	—	—

Example usage in KYRO:

```
app = FastAPI(title=settings.app_name, version="0.1.0")
app.include_router(auth.router)  # Mounts /api/v1/auth/*
```

SQLAlchemy (>=2.0.30)

What it is: An Object-Relational Mapper (ORM) that lets Python code work with database tables as Python classes, without writing raw SQL.

Simple explanation: Instead of writing `SELECT * FROM customers WHERE id = '...'`, you write `db.get(Customer, customer_id)` in Python.

Why it is used: Provides database abstraction, connection pooling, and type-safe queries. Version 2.0 introduced a new "mapped column" style that is cleaner.

Configuration in KYRO:

```
engine = create_engine(
    settings.database_url,
    pool_pre_ping=True,   # Check connection health before use
```

```
    pool_size=10,          # Maintain 10 persistent connections
    max_overflow=20,       # Allow up to 20 additional burst connections
)
SessionLocal = sessionmaker(
    bind=engine,
    autocommit=False,      # Manual commit required
    autoflush=False,       # Don't flush on every query
    expire_on_commit=False # Don't reload objects after commit
)
```

Connection Pool Explained:

- `pool_size=10` : 10 permanent DB connections always open
- `max_overflow=20` : Up to 20 additional connections during peak load
- `pool_pre_ping=True` : Sends a `SELECT 1` before using each connection to detect stale connections

---

Pydantic (>=2.9.0) + pydantic-settings

What it is: A data validation library using Python type annotations.

Why it is used: Every API request body and response is validated by Pydantic. If a client sends wrong data types, Pydantic catches it before the code runs.

KYRO Settings Example:

```
class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_file=".env", extra="ignore")
    database_url: str = "postgresql+psycopg://..."
    secret_key: str = "change-me-in-production"
    access_token_expire_minutes: int = 30
```

Loaded once and cached:

```
@lru_cache(maxsize=1)  # Only computed once; reused for all requests
def get_settings() -> Settings:
    return Settings()
```

PyJWT (>=2.9.0) + Passlib + bcrypt

What they are: Libraries for generating/verifying JSON Web Tokens and hashing passwords.

JWT in KYRO:

```
payload = {
    "sub": user_id,      # Subject (who is this token for)
    "role": "ANALYST",   # RBAC role embedded
    "type": "access",    # access or refresh
    "iat": issued_at,    # Issued at timestamp
    "exp": expiry        # Expiry timestamp
}
token = jwt.encode(payload, secret_key, algorithm="HS256")
```

Password hashing:

```
_pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")
hashed = _pwd_context.hash("my_password")  # Produces $2b$12$...
valid = _pwd_context.verify("my_password", hashed)  # True/False
```

3.2 Database

PostgreSQL 16 (Alpine)

What it is: The world's most advanced open-source relational database. Supports ACID transactions, JSON storage, advanced indexing, and extensibility.

Why it is used:

- ACID compliance for financial data integrity
- JSONB for flexible risk_flags and metadata storage
- BRIN indexes for time-series append-only data (transactions)
- GIN indexes for JSONB containment queries

Advanced Features Used:
Feature	Usage in KYRO
JSONB	`risk_flags`, `ml_explanation`, `customer_metadata`
BRIN Index	`transactions.transaction_date`, `audit_logs.performed_at`
GIN Index	`transactions.risk_flags` (for `@>` containment)
CHECK Constraints	Enforce enum values at DB level
`func.percentile_cont()`	Compute p50/p90/p99 for feature engineering

| `ON DELETE CASCADE` | Orphan cleanup when parent deleted |
| UUID Primary Keys | Globally unique IDs across distributed systems |

Version: 16-alpine (lightweight container image)

Redis 7 (Alpine)

What it is: An in-memory data store used as a cache, message broker, and result backend.

Why Redis, not Memcached?

- Redis supports data persistence (Memcached is memory-only)
- Redis supports pub/sub and lists (needed by Celery)
- Redis handles more complex data structures (hashes, sorted sets)

Dual Role in KYRO:

1. **Celery Broker:** Stores task queue messages (pending ETL/training jobs)
2. **Feature Cache:** Stores global amount statistics (mean, std, p50/p90/p99) with 1-hour TTL to avoid recomputing on every request

```
GLOBAL_STATS_CACHE_KEY = "ml:global_amount_stats"
GLOBAL_STATS_TTL_SECONDS = 3600  # 1 hour
```

Port Mapping: Host 6380 → Container 6379 (avoids conflict with local Redis)

3.3 Machine Learning

scikit-learn (>=1.4.0)

What it is: The most widely used Python ML library. Provides implementations of Random Forest, Isolation Forest, StandardScaler, and all evaluation metrics.

Why sklearn over PyTorch/TensorFlow?

- Tabular financial data doesn't benefit from deep learning
- Random Forest is more interpretable and auditable
- SHAP TreeExplainer is optimized for sklearn tree ensembles
- No GPU required — runs in CPU containers

Models used:

1. `RandomForestRegressor` — Predicts continuous risk score (0–100)
2. `RandomForestClassifier` — Predicts anomaly probability (0.0–1.0)
3. `IsolationForest` — Unsupervised outlier detection

SHAP (>=0.45.0)

What it is: SHapley Additive exPlanations — a game theory-based method to explain any ML model's predictions.

Simple explanation: SHAP answers "why did the model give this score?" by measuring how much each feature contributed to the prediction.

Why SHAP in KYRO:

- Regulatory requirement: ML decisions must be explainable
- `TreeExplainer` is exact (not approximate) for tree-based models
- Converts raw SHAP values into analyst-readable English descriptions

Example output:

```
{
  "top_features": [
    {"feature": "amount_zscore", "impact": 15.3, "direction":
"INCREASES_RISK",
     "description": "Amount deviates significantly from the global baseline"},
    {"feature": "high_risk_country_flag", "impact": 12.1, "direction":
"INCREASES_RISK",
     "description": "Transaction involves a high-risk jurisdiction"}
  ],
  "summary": "Amount deviates significantly from the global baseline;
Transaction involves a high-risk jurisdiction"
}
```

Pandas (>=2.1.0) + NumPy (>=1.26.0)

Pandas: DataFrame library for tabular data manipulation during feature engineering and training data preparation.

NumPy: Numerical computation library underlying scikit-learn's mathematical operations.

3.4 Async Task Processing

Celery (>=5.4.0)

What it is: A distributed task queue. Allows Python functions to be executed asynchronously in background workers, separate from the API server.

Why Celery?

- ML training can take minutes — blocking the API would timeout
- Daily ETL scan of unscored transactions runs at 02:00 UTC (background)
- Celery Beat provides cron-like scheduling

Two process types:

- `celery_worker` : Executes tasks from the queue
- `celery_beat` : Schedules tasks on a cron schedule

Scheduled tasks:

```
beat_schedule = {
    "run-daily-etl-pipeline": {
        "task": "app.tasks.etl_tasks.run_daily_etl_pipeline",
        "schedule": crontab(hour=2, minute=0),  # 02:00 UTC daily
    },
    "check-ml-retraining-weekly": {
        "task": "app.tasks.ml_tasks.check_retraining_task",
        "schedule": crontab(day_of_week=0, hour=3, minute=0),  # Sunday 03:00
UTC
    },
}
```

3.5 Docker

What it is: A containerization platform that packages applications with all their dependencies.

Why Docker?

- Eliminates "it works on my machine" problems

- Consistent environment from development to production
- Easy scaling by adding more container replicas
- Health checks ensure services only start when dependencies are ready

7 Services in KYRO:

Service	Image	Port	Purpose
postgres	postgres:16-alpine	5434:5432	Main database
pgadmin	dpage/pgadmin4	5050:80	DB admin UI
pipeline	Custom (Dockerfile.pipeline)	—	ETL data generator
redis	redis:7-alpine	6380:6379	Broker/cache
api	Custom (Dockerfile.api)	8000:8000	FastAPI server
celery_worker	Custom (Dockerfile.api)	—	Task executor
celery_beat	Custom (Dockerfile.api)	—	Task scheduler

3.6 Technology Alternatives Not Selected

Technology Considered	Reason Not Selected
Django REST Framework	Too heavyweight, slower, less suitable for microservice
MongoDB	Lacks ACID guarantees needed for financial data
MySQL	Fewer advanced features than PostgreSQL (no BRIN/GIN)
RabbitMQ	Redis already in stack; added complexity not justified
MLflow	Requires extra service; simple JSON registry is sufficient
Kafka	Overkill for current transaction volume; future enhancement
TensorFlow	Tabular data doesn't need deep learning; sklearn interpretable
JWT with RS256	HS256 simpler for single-service; RS256 for distributed auth

---

4. Folder Structure

4.1 Root Directory

```
KYRO_NEW/
├── app/                    ← FastAPI application (main product code)
├── pipeline/               ← ETL data pipeline
├── docker/                 ← Docker configuration files
├── models/                 ← Trained ML model .pkl files
├── scripts/                ← Utility scripts
├── tests/                  ← Test suite
├── generator/              ← Synthetic data generator
├── output/                 ← Pipeline output files
├── logs/                   ← Application logs
├── artifacts/              ← Pipeline artifacts
├── docker-compose.yml      ← Docker service orchestration
├── .env.example            ← Environment variable template
├── requirements-api.txt    ← API service dependencies
├── requirements-ml.txt     ← ML engine dependencies
├── requirements-pipeline.txt ← Pipeline dependencies
├── requirements.txt        ← Top-level requirements
├── pytest.ini              ← Test configuration
├── alembic.ini             ← Database migration config
├── run.sh                  ← Startup script
├── PHASE2.txt              ← Phase 2 implementation notes
└── phase1.txt              ← Phase 1 implementation notes
```

4.2 App Directory (FastAPI Application)

```
app/
├── __init__.py             ← Package marker
├── main.py                 ← FastAPI app factory, CORS, router inclusion
├── config.py               ← Pydantic Settings loader (lru_cache)
├── database.py             ← SQLAlchemy engine, session factory, get_db dep
├── deps.py                 ← FastAPI dependencies (auth, RBAC, pagination)
│
├── models/                 ← SQLAlchemy ORM models (maps to DB tables)
│   ├── __init__.py         ← Exports all models for Alembic autodiscovery
│   ├── base.py             ← Base class, SCHEMA constant, mixins
│   ├── user.py             ← Users table (authentication)
│   ├── customer.py         ← Customers, risk profiles, KYC, PEP/sanctions
│   ├── account.py          ← Accounts, metadata, balance history
│   ├── transaction.py      ← Transactions, counterparties, risk flags
│   ├── alert.py            ← Alerts (analyst work queue)
│   ├── audit.py            ← Audit logs (immutable)
│   └── ml_score.py         ← ML scoring results
│
├── schemas/                ← Pydantic schemas (request/response contracts)
│   ├── auth.py             ← Token, UserCreate, UserOut, RefreshRequest
```

```
│   ├── customer.py         ← CustomerCreate, CustomerOut
│   ├── account.py          ← AccountCreate, AccountOut
│   ├── transaction.py      ← TransactionCreate, TransactionOut, risk schemas
│   ├── alert.py            ← AlertOut, AlertUpdate
│   ├── common.py           ← Page[T] generic pagination schema
│   └── ml.py               ← ML scoring request/response schemas
│
├── routers/                ← FastAPI route handlers (HTTP endpoints)
│   ├── __init__.py
│   ├── auth.py             ← /api/v1/auth/* (register, login, logout,
refresh, me)
│   ├── customers.py        ← /api/v1/customers/* (CRUD + risk profile)
│   ├── accounts.py         ← /api/v1/accounts/* (CRUD + balance)
│   ├── transactions.py     ← /api/v1/transactions/* (ingest, list, risk)
│   ├── alerts.py           ← /api/v1/alerts/* (list, update, resolve)
│   ├── kyc.py              ← /api/v1/kyc/* (reviews, screenings)
│   └── ml.py               ← /api/v1/ml/* (score, train, models, performance)
│
├── services/               ← Business logic (not HTTP-specific)
│   ├── rules_engine.py     ← R001-R010 deterministic risk rules
│   ├── alert_service.py    ← ML score → Alert routing logic
│   ├── audit_service.py    ← Audit log writer
│   ├── customer_service.py ← Customer risk profile update
│   ├── feedback_service.py ← Analyst feedback → performance metrics
│   └── retraining_service.py ← Auto-retraining decision logic
│
├── ml/                     ← Machine Learning engine
│   ├── __init__.py
│   ├── features/           ← Feature engineering
│   │   ├── engineer.py     ← compute_transaction_features() (40+ features)
│   │   ├── customer_profile.py ← Behavioral baseline computation
│   │   └── feature_store.py ← Redis cache get/set for features
│   ├── models/             ← ML model class definitions
│   │   ├── risk_scorer.py  ← RiskScorerModel (RF Regressor)
│   │   ├── anomaly_classifier.py ← AnomalyClassifier (RF Classifier)
│   │   └── isolation_detector.py ← UnsupervisedAnomalyDetector (Isolation
Forest)
│   ├── scoring/            ← Scoring orchestration
│   │   ├── real_time_scorer.py ← Single-transaction scoring with SHAP
│   │   └── batch_scorer.py ← Multi-transaction bulk scoring
│   ├── training/           ← Model training
│   │   ├── pipeline.py     ← Training orchestration (train + register)
│   │   └── trainer.py      ← Data pull + train_all() function
│   ├── registry/           ← Model version management
│   │   └── model_registry.py ← Save/load/version/A-B routing
│   └── explainability/     ← SHAP explainability
```

```
│       └── shap_explainer.py ← ExplainabilityEngine (TreeExplainer)
│
├── tasks/                    ← Celery background tasks
│   ├── celery_app.py         ← Celery app config + beat schedule
│   ├── etl_tasks.py          ← Daily ETL pipeline tasks
│   └── ml_tasks.py           ← ML training Celery tasks
│
└── utils/                    ← Shared utilities
    └── security.py           ← hash_password, verify_password, JWT
create/decode
```

4.3 File-by-File Explanation

`app/main.py` — Application Entry Point

```
# Line 1–4: Module docstring
# Line 7–8: Import FastAPI and CORS middleware
# Line 10–11: Import settings and all 7 routers
# Line 13: Call get_settings() – reads .env file once
# Line 15: Create FastAPI app instance
# Line 17–23: Add CORS middleware (allow all origins for dev)
# Lines 25–31: Mount all 7 API routers
# Lines 34–36: /api/v1/health – simple health check endpoint
```

Note on CORS: `allow_origins=["*"]` is fine for development but must be restricted to known frontend domains in production.

`app/config.py` — Settings Management

```
class Settings(BaseSettings):
    # model_config: Reads from .env file, ignores unknown vars
    # All fields have defaults so the app starts without .env
    # lru_cache(maxsize=1): Settings computed once per process lifetime
```

Why `@lru_cache`? Reading and parsing the `.env` file on every API request would be wasteful. `lru_cache(maxsize=1)` means `get_settings()` is computed once and the result is cached in memory for the entire process lifetime.

`app/database.py` — Database Connection

```
# create_engine: Establishes connection to PostgreSQL
# pool_pre_ping=True: Tests connection health before use
# pool_size=10: 10 persistent connections
```

```
# max_overflow=20: Up to 30 total in burst
# expire_on_commit=False: SQLAlchemy objects remain usable after commit
# get_db(): Context-managed DB session for FastAPI Depends()
```

`app/deps.py` — FastAPI Dependencies

```
# oauth2_scheme: Extracts Bearer token from Authorization header
# get_current_user(): Validates JWT, loads user from DB
# require_role(*roles): Returns 403 if user's role not in allowed roles
# PaginationParams: Extracts ?page=1&page_size=20 from query string
```

RBAC Flow:

```
Request arrives
    ↓
oauth2_scheme extracts Bearer token
    ↓
decode_token() decodes JWT
    ↓
db.get(User, uuid) loads user
    ↓
if user.role not in allowed_roles → 403 Forbidden
    ↓
else → proceed to route handler
```

4.4 Pipeline Directory

```
pipeline/
├── run_pipeline.py         ← Main pipeline entry point (38KB)
├── __init__.py
├── ingestion/              ← Data ingestion from source systems
├── cleaning/               ← Data cleaning and deduplication
├── transformation/         ← Data transformation logic
├── feature_engineering/    ← Feature computation for ETL
├── validation/             ← Data quality validation
├── quality/                ← Data quality metrics
├── loaders/                ← Database load operations
├── models/                 ← Pipeline data models
├── migrations/             ← SQL DDL migration scripts
│   ├── 001_create_schemas_tables.sql
│   ├── 002_create_indexes.sql
```

```
│   ├── 003_triggers_procedures_views.sql
│   └── 004_security_roles.sql
├── monitoring/             ← Pipeline health monitoring
├── security/               ← Data security utilities
├── core/                   ← Core pipeline utilities
├── config/                 ← Pipeline configuration
└── scripts/                ← Helper scripts
```

4.5 Docker Directory

```
docker/
├── Dockerfile.api          ← API + Celery worker/beat image
├── Dockerfile.pipeline     ← ETL pipeline runner image
└── postgresql.conf         ← Custom PostgreSQL tuning
```

Dockerfile.api builds one image that is reused by three services:

- `api` (uvicorn server)
- `celery_worker` (worker process)
- `celery_beat` (scheduler)
  This is a best practice: same code, different entry commands.

4.6 Models Directory

```
models/                     ← ML model artifacts (mounted as Docker volume)
├── registry.json           ← Active/candidate version pointers
├── risk_scorer_v1.pkl      ← Trained RiskScorerModel (pickle)
├── anomaly_classifier_v1.pkl
└── isolation_detector_v1.pkl
```

Important: The `models/` directory is a Docker bind-mount, meaning both the API container and the host filesystem share the same files. This allows model artifacts trained by the ML engine to be immediately available to the scoring API.

---

5. Complete Workflow

5.1 Transaction Ingestion & Scoring Workflow

This is the most important workflow. Here is exactly what happens when a transaction is submitted:

```
Client sends: POST /api/v1/transactions
  Body: {customer_id, account_id, amount, transaction_type, ...}
     |
     ▼
Step 1: CORS Middleware checks Origin header
     |
     ▼
Step 2: oauth2_scheme extracts Bearer token from Authorization header
     |
     ▼
Step 3: get_current_user() validates JWT
  • decode_token() decodes HS256 token
  • Checks "type" == "access"
  • Loads User from database by user.id (UUID)
  • If inactive → 401 Unauthorized
     |
     ▼
Step 4: Pydantic validates request body (TransactionCreate schema)
  • amount > 0 required
  • transaction_type must be in allowed enum values
  • customer_id and account_id must be valid UUIDs
     |
     ▼
Step 5: _ingest_one() business logic
  a. db.get(Customer, data.customer_id) → 404 if not found
  b. db.get(Account, data.account_id) → 404 if not found
  c. account.customer_id == customer.id → 400 if mismatch
  d. Create Transaction ORM object, db.add(txn), db.flush()
     |
     ▼
Step 6: apply_to_transaction() — RULES ENGINE
  R001: amount > 10,000? → MEDIUM severity
  R002: daily_count > 5? → MEDIUM severity
  R003: hourly_count > 3? → LOW severity
  R004: high_risk_country? → HIGH severity
  R005: pep_flag? → HIGH severity
  R006: sanctions_flag? → CRITICAL severity
  R007: new_counterparty? → LOW severity
  R008: weekend? → LOW severity
  R009: round_amount >= 1000? → MEDIUM severity
  R010: rapid_succession within 60s? → HIGH severity
     |
     ▼
Step 7: score_from_rules() — compute weighted sum
  LOW=10, MEDIUM=25, HIGH=50, CRITICAL=90
  Score = min(100, sum of triggered rule weights)
```

```
     |
     ▼
Step 8: recommended_action_for()
  CRITICAL rule → "SAR"
  score >= 75 → "ENHANCED_DUE_DILIGENCE"
  score >= 50 → "REVIEW"
  else → None (no alert)
     |
     ▼
Step 9: If action is not None → create Alert record
  Alert.status = "OPEN"
  Alert.alert_type = "SANCTIONS" / "PEP" / "BEHAVIORAL_ANOMALY"
     |
     ▼
Step 10: log_action() — write AuditLog record
  entity_type="TRANSACTION", action="CREATE"
     |
     ▼
Step 11: db.commit() — all changes written atomically
     |
     ▼
Step 12: Return TransactionOut response (201 Created)
  {id, customer_id, amount, transaction_type, risk_score, risk_flags, ...}
```

5.2 ML Scoring Workflow

```
Client sends: POST /api/v1/ml/score-transaction
  Body: {transaction_id: "uuid..."}
     |
     ▼
Step 1: Auth (same as above — JWT required)
     |
     ▼
Step 2: db.get(Transaction, data.transaction_id)
        db.get(Customer, txn.customer_id)
        → 404 if either not found
     |
     ▼
Step 3: RealTimeScorer().score_transaction(db, txn, customer)
     |
     ▼
Step 4: _load("risk_scorer")
  • registry.resolve_serving_version("risk_scorer")
    → randomly routes to candidate (if traffic_pct% chance)
    → or returns active version
```

```
  • registry.load_model(name, version) → unpickle .pkl file
     |
     ▼
Step 5: compute_transaction_features(db, txn, customer)
  → Queries 90-day transaction history (1 SQL query)
  → Computes 40+ features (rolling stats, velocity, z-scores, etc.)
  → Queries global stats from Redis cache (or PostgreSQL)
  → Returns dict[str, float]
     |
     ▼
Step 6: X = pd.DataFrame([features])
  risk_scorer.predict(X) → rf_risk_score (0-100 float)
  anomaly_classifier.predict_proba(X) → anomaly_probability (0.0-1.0)
  isolation_detector.anomaly_score(X) → raw outlier score
  isolation_to_0_100(raw) → isolation_score (0-100)
     |
     ▼
Step 7: Combined score formula:
  combined = 0.50 × rf_risk_score
           + 0.35 × (anomaly_probability × 100)
           + 0.15 × isolation_score
  combined = clamp(combined, 0.0, 100.0)
     |
     ▼
Step 8: anomaly_flag determination:
  flag = (anomaly_probability > 0.5) OR (combined_score >= 71)
     |
     ▼
Step 9: SHAP Explanation
  ExplainabilityEngine(risk_scorer, feature_names)
  explainer.shap_values(X_scaled) → per-feature SHAP values
  Top 5 by |impact| → format as {feature, impact, direction, description}
     |
     ▼
Step 10: Persist MLScore record to database
  {transaction_id, rf_risk_score, anomaly_probability,
   isolation_score, combined_score, anomaly_flag, explanation, features}
     |
     ▼
Step 11: AlertRouter().route()
  combined_score <= 30 → no alert (log only)
  30 < score <= 70 → BATCH_REVIEW alert
  score > 70 → IMMEDIATE_REVIEW alert
     |
     ▼
Step 12: db.commit()
```

```
     |
     ▼
Step 13: Return ScoreTransactionResponse
  {risk_score, anomaly_flag, confidence, ml_explanation,
   alert_created, alert_id, recommended_action}
```

5.3 User Registration & Login Workflow

```
POST /api/v1/auth/register
  {username, email, password, full_name}
     |
     ▼
Check username unique → 409 Conflict if taken
Check email unique → 409 Conflict if taken
hash_password(password) → bcrypt hash
Create User(role="ANALYST") ← Always lowest privilege
db.commit()
Return UserOut (201 Created)

POST /api/v1/auth/login
  Form: username=..., password=...
     |
     ▼
db.query(User).filter(username == ...).first()
verify_password(plain, hashed) → True/False
if user.is_active == False → 403 Forbidden
create_access_token(user.id, user.role) → JWT (30 min expiry)
create_refresh_token(user.id, user.role) → JWT (7 day expiry)
user.last_login = now()
db.commit()
Return Token {access_token, refresh_token, token_type="bearer"}
```

5.4 Background ETL Workflow (Daily @ 02:00 UTC)

```
Celery Beat triggers run_daily_etl_pipeline at 02:00 UTC
     |
     ▼
extract_from_source_system()
  → SELECT id FROM transactions WHERE risk_flags IS NULL
  → Returns list of unscored transaction UUIDs
     |
     ▼
validate_transaction_data(ids)
  → For each ID: check transaction + customer exist
```

```
   → Drop orphaned IDs
   → Returns valid list
      |
      ▼
transform_and_load(valid_ids)
   → For each valid ID:
       apply_to_transaction(db, txn, customer)
       scored += 1
   → db.commit()
   → Returns {"scored": N}
```

---

6. Frontend Documentation

> **Assumption:** The KYRO system is a backend-only API platform. No custom frontend application is implemented in the current codebase. The interactive interface provided is FastAPI's **Swagger UI** at `http://localhost:8000/docs`.

6.1 Swagger UI (Interactive API Documentation)

FastAPI auto-generates a full Swagger UI interface that serves as the operational frontend for developers and testers.

Access: `http://localhost:8000/docs`

Available sections in Swagger:

- **auth** — Register, Login, Logout, Refresh, Me
- **customers** — CRUD customer management
- **accounts** — Account management
- **transactions** — Transaction ingestion and risk retrieval
- **alerts** — Alert work queue management
- **kyc** — KYC reviews and PEP/sanctions screening
- **ml** — ML scoring, training, and model management
- **health** — Health check

6.2 How to Use Swagger UI

1. Open `http://localhost:8000/docs`
2. Click **POST /api/v1/auth/register** → Register a user
3. Click **POST /api/v1/auth/login** → Get access token
4. Click **Authorize** button (top right) → Enter `Bearer <access_token>`

5. Now all authenticated endpoints are unlocked

6.3 Future Frontend Design Recommendation

A production frontend should include:

Page	Components	Data Source
Login	Form (username/password)	POST /api/v1/auth/login
Dashboard	Risk overview chart, alert count	GET /api/v1/alerts, GET /api/v1/ml/performance
Customers	Paginated table with risk levels	GET /api/v1/customers
Customer Detail	Risk profile, transaction history	GET /api/v1/customers/{id}
Transactions	Filtered table with risk scores	GET /api/v1/transactions
Transaction Detail	Risk flags, ML explanation	GET /api/v1/transactions/{id}/risk
Alerts	Work queue (OPEN/ASSIGNED/IN_REVIEW)	GET /api/v1/alerts
Alert Detail	SHAP explanation, recommended action	GET /api/v1/alerts/{id}
KYC Reviews	Review scheduling, status tracking	GET /api/v1/kyc/reviews
ML Models	Version status, A/B testing	GET /api/v1/ml/models
ML Performance	Precision, false positive rate chart	GET /api/v1/ml/performance

Recommended tech stack: React 18 + TypeScript + Recharts + React Query

---

7. Backend Documentation

7.1 Authentication Endpoints

POST /api/v1/auth/register

Purpose: Create a new user account with the lowest-privilege role (ANALYST).

Why always ANALYST? Self-service registration could be exploited to grant admin rights if the role was user-controlled. COMPLIANCE_OFFICER and ADMIN roles must be granted by an existing admin via direct database update.

Request:

```
{
  "username": "john_analyst",
  "email": "john@bank.com",
  "password": "SecurePass123!",
  "full_name": "John Smith"
}
```

Response (201 Created):

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "username": "john_analyst",
  "email": "john@bank.com",
  "full_name": "John Smith",
  "role": "ANALYST",
  "is_active": true,
  "created_at": "2026-07-30T12:00:00Z"
}
```

Error Responses:

- `409 Conflict` — Username already taken
- `409 Conflict` — Email already registered
- `422 Unprocessable Entity` — Validation error (missing fields)

Internal Flow:

1. `db.query(User).filter(User.username == data.username).first()` → duplicate check
2. `hash_password(data.password)` → bcrypt hash
3. `User(role="ANALYST")` → hardcoded lowest privilege
4. `db.add(user)`, `db.commit()`, `db.refresh(user)`

POST /api/v1/auth/login

Purpose: Authenticate user credentials and return JWT access + refresh tokens.

Request (OAuth2 form, not JSON):

```
Content-Type: application/x-www-form-urlencoded
Body: username=john_analyst&password=SecurePass123!
```

Response (200 OK):

```
{
  "access_token": "eyJhbGciOiJIUzI1NiJ9...",
  "refresh_token": "eyJhbGciOiJIUzI1NiJ9...",
  "token_type": "bearer"
}
```

Error Responses:

- `401 Unauthorized` — Wrong username or password
- `403 Forbidden` — User is inactive

POST /api/v1/auth/refresh

Purpose: Exchange a valid refresh token for new access + refresh token pair.

Request:

```
{"refresh_token": "eyJhbGciOiJIUzI1NiJ9..."}
```

Response: Same as login response.

Validation:

- Decodes token with `decode_token()`
- Checks `claims["type"] == "refresh"` (rejects access tokens)
- Loads user and checks `is_active`

GET /api/v1/auth/me

Purpose: Return the currently authenticated user's profile.

Headers:

```
Authorization: Bearer <access_token>
```

Response:

```
{
  "id": "...",
  "username": "john_analyst",
  "role": "ANALYST",
  "is_active": true
}
```

7.2 Customer Endpoints

POST /api/v1/customers

Purpose: Create a new customer profile.

Required Role: Any authenticated user

Request:

```
{
  "full_name": "Alice Johnson",
  "email": "alice@example.com",
  "phone": "+1-555-0100",
  "date_of_birth": "1985-03-15",
  "country": "US",
  "residency_country": "US",
  "customer_type": "INDIVIDUAL",
  "kyc_status": "PENDING"
}
```

Response (201 Created):

```
{
  "id": "...",
  "full_name": "Alice Johnson",
  "email": "alice@example.com",
  "kyc_status": "PENDING",
  "risk_level": "LOW",
  "risk_score": 0,
```

```
  "pep_flag": false,
  "sanctions_flag": false,
  "created_at": "2026-07-30T12:00:00Z"
}
```

GET /api/v1/customers

Purpose: Paginated list of all customers.

Query Parameters:

- `page` (default: 1) — Page number
- `page_size` (default: 20, max: 200) — Items per page
- `risk_level` (optional) — Filter: LOW, MEDIUM, HIGH
- `kyc_status` (optional) — Filter: PENDING, VERIFIED, REJECTED, UNDER_REVIEW

Response:

```
{
  "items": [...],
  "total": 1234,
  "page": 1,
  "page_size": 20
}
```

7.3 Transaction Endpoints

POST /api/v1/transactions

Purpose: Ingest a single transaction. Immediately scored by rules engine synchronously.

Request:

```
{
  "customer_id": "550e8400-...",
  "account_id": "660e8400-...",
  "transaction_date": "2026-07-30T10:00:00Z",
  "transaction_type": "TRANSFER",
  "amount": 15000.00,
  "currency": "USD",
```

```
  "meta_counterparty": "Foreign Corp Ltd",
  "meta_country": "IR",
  "meta_destination_country": "IR"
}
```

Response (201 Created):

```
{
  "id": "...",
  "amount": 15000.00,
  "transaction_type": "TRANSFER",
  "risk_score": 140,
  "risk_flags": {"triggered_rules": ["R001", "R004"]},
  "currency": "USD",
  "created_at": "2026-07-30T10:00:00.123Z"
}
```

Note on risk_score: After rules engine, `min(100, 25+50)=75`. But the raw sum before capping shows which rules fired.

POST /api/v1/transactions/batch

Purpose: Ingest multiple transactions atomically. All succeed or all fail.

Request:

```
{
  "transactions": [
    {"customer_id": "...", "account_id": "...", "amount": 100.00, ...},
    {"customer_id": "...", "account_id": "...", "amount": 5000.00, ...}
  ]
}
```

GET /api/v1/transactions/{id}/risk

Purpose: Retrieve the rule-based risk assessment for a transaction.

Response:

```
{
  "transaction_id": "...",
  "risk_score": 75,
  "risk_flags": {"triggered_rules": ["R001", "R004"]},
```

```
  "triggered_rules": ["R001", "R004"]
}
```

GET /api/v1/transactions/{id}/flags

Purpose: Get detailed risk flag records for a transaction.

Response:

```
[
  {
    "id": "...",
    "flag_type": "R001",
    "flag_description": "Amount Threshold: Transaction 15000.00 > 10000",
    "flag_severity": "MEDIUM",
    "triggered_at": "2026-07-30T10:00:00Z",
    "triggered_by": "RULES_ENGINE"
  },
  {
    "id": "...",
    "flag_type": "R004",
    "flag_description": "High Risk Country: Counterparty in sanctioned/high-
risk country",
    "flag_severity": "HIGH",
    "triggered_at": "2026-07-30T10:00:00Z",
    "triggered_by": "RULES_ENGINE"
  }
]
```

7.4 ML Endpoints

POST /api/v1/ml/score-transaction

Purpose: Apply the full ML ensemble to a transaction. Transaction must already exist in the database (created via POST /transactions first).

Request:

```
{"transaction_id": "550e8400-..."}
```

Response:

```
{
  "transaction_id": "550e8400-...",
  "risk_score": 82.5,
  "anomaly_flag": true,
  "confidence": 0.89,
  "ml_explanation": {
    "top_features": [
      {"feature": "amount_zscore", "impact": 18.5, "direction":
"INCREASES_RISK",
       "description": "Amount deviates significantly from the global
baseline"},
      {"feature": "high_risk_country_flag", "impact": 12.1, "direction":
"INCREASES_RISK",
       "description": "Transaction involves a high-risk jurisdiction"},
      {"feature": "pep_flag", "impact": 8.3, "direction": "INCREASES_RISK",
       "description": "Customer is a Politically Exposed Person"}
    ],
    "summary": "Amount deviates significantly from the global baseline;
Transaction involves a high-risk jurisdiction; Customer is a Politically
Exposed Person",
    "base_value": 35.0,
    "prediction": 82.5
  },
  "alert_created": true,
  "alert_id": "660e8400-...",
  "recommended_action": "IMMEDIATE_REVIEW"
}
```


Error Responses:

- `404 Not Found` — Transaction or customer not found
- `409 Conflict` — Models not trained yet (no .pkl files)

POST /api/v1/ml/train

Purpose: Train all three ML models on transaction data from the past 365 days.

Required Role: ADMIN only

Request:


```
{
  "run_async": true,
  "as_candidate": true,
  "candidate_traffic_pct": 10.0,
```

```
  "limit": null
}
```

Parameters explained:

- `run_async`: If true, dispatches to Celery worker and returns task_id immediately
- `as_candidate`: If true, new model goes to candidate slot (A/B testing). If false, directly becomes active
- `candidate_traffic_pct`: Percentage of scoring requests routed to the candidate model
- `limit`: Optional limit on training samples (for testing with small datasets)

Response (async):

```
{"status": "QUEUED", "task_id": "abc123..."}
```

Response (sync):

```
{
  "status": "COMPLETED",
  "versions": {
    "risk_scorer": 2,
    "anomaly_classifier": 2,
    "isolation_detector": 2
  },
  "metrics": {
    "mse": 45.2,
    "mae": 6.1,
    "r2": 0.87,
    "anomaly_accuracy": 0.93,
    "anomaly_precision": 0.88,
    "anomaly_recall": 0.91,
    "anomaly_f1": 0.89,
    "anomaly_auc": 0.95
  }
}
```

GET /api/v1/ml/models

Purpose: List all model versions and their current active/candidate status.

Response:

```
[
  {
    "name": "risk_scorer",
    "active_version": 1,
    "candidate_version": 2,
    "candidate_traffic_pct": 10.0,
    "available_versions": [1, 2]
  },
  {
    "name": "anomaly_classifier",
    "active_version": 1,
    "candidate_version": null,
    "candidate_traffic_pct": 0,
    "available_versions": [1]
  }
]
```


GET /api/v1/ml/performance

Purpose: Evaluate ML model precision from analyst feedback on resolved alerts.

Query Parameters:

- `window_days` (default: 30) — Look back N days for resolved alerts

Response:


```
{
  "precision": 0.78,
  "false_positive_rate": 0.22,
  "total_reviewed": 145,
  "window_days": 30
}
```


How precision is calculated:

```
precision = true_positives / total_reviewed
false_positive_rate = false_positives / total_reviewed
```

Where `false_positives` = alerts marked `is_false_positive = True` by analysts.

7.5 Alert Endpoints

GET /api/v1/alerts

Purpose: List alerts for analyst review work queue.

Query Parameters:

- `status` — Filter: OPEN, ASSIGNED, IN_REVIEW, RESOLVED, ESCALATED
- `customer_id` — Filter by customer

PATCH /api/v1/alerts/{id}

Purpose: Update alert status, assign to analyst, or resolve with notes.

Request:

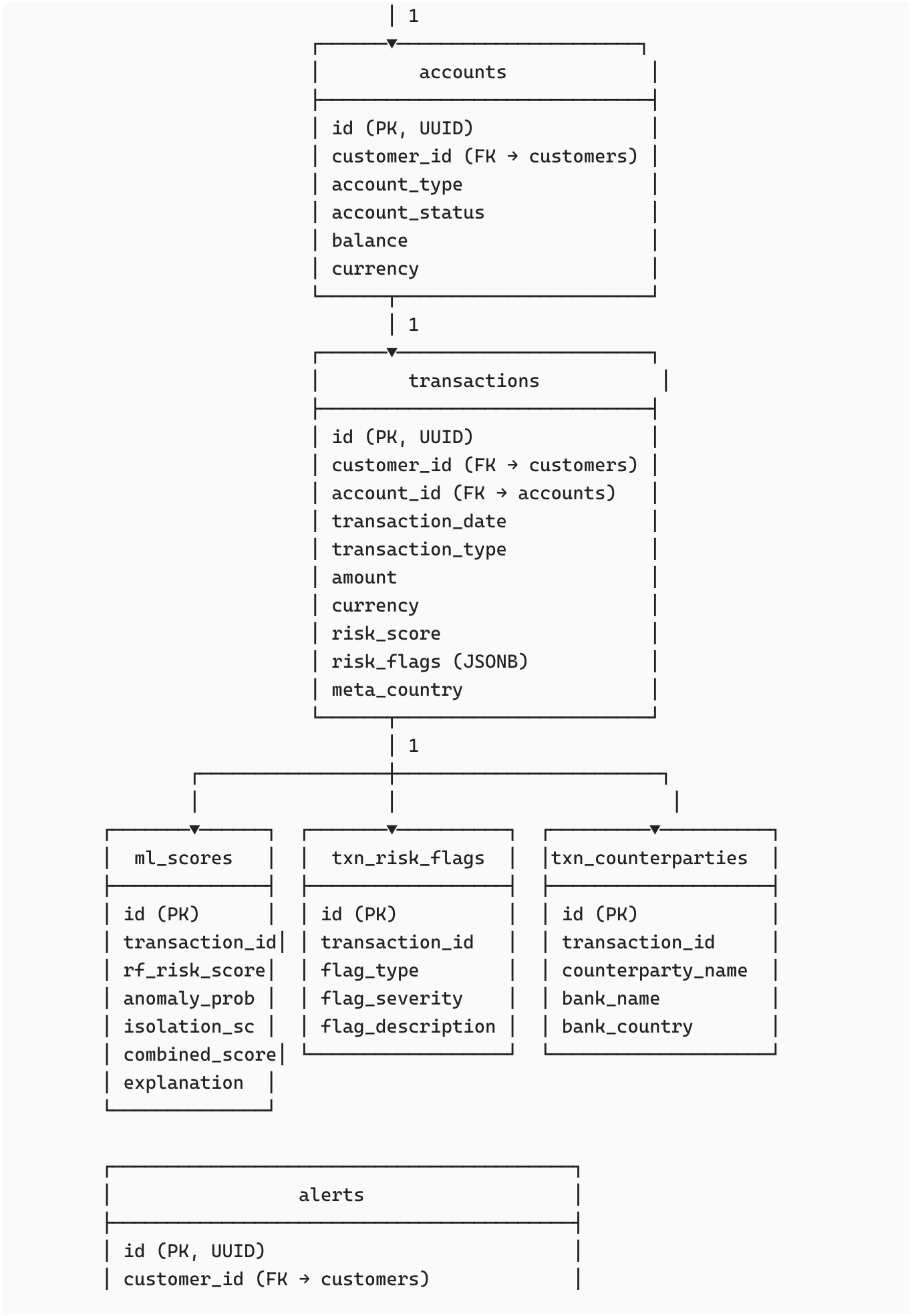
```
{
  "status": "RESOLVED",
  "resolution_notes": "Investigated and cleared - legitimate business
transaction",
  "is_false_positive": true
}
```

This feedback is collected by `feedback_service.collect_feedback()` and used for model performance evaluation.

---

8. Database Documentation

8.1 ER Diagram


1
accounts
id (PK, UUID)
customer_id (FK → customers)
account_type
account_status
balance
currency
1
transactions
id (PK, UUID)
customer_id (FK → customers)
account_id (FK → accounts)
transaction_date
transaction_type
amount
currency
risk_score
risk_flags (JSONB)
meta_country
1
ml_scores
id (PK)
transaction_id
rf_risk_score
anomaly_prob
isolation_sc
combined_score
explanation
txn_risk_flags
id (PK)
transaction_id
flag_type
flag_severity
flag_description
txn_counterparties
id (PK)
transaction_id
counterparty_name
bank_name
bank_country
alerts
id (PK, UUID)
customer_id (FK → customers)

```
        | alert_type                                |
        | risk_score                                |
        | confidence                                |
        | triggered_rules (JSONB)                   |
        | ml_explanation (JSONB)                    |
        | recommended_action                        |
        | status                                    |
        | assigned_to (UUID, nullable - no FK)      |
        | is_false_positive                         |
        └──────────────────────────────────────────┘


        ┌──────────────────────────────────────────┐
        |              audit_logs                  |
        ├──────────────────────────────────────────┤
        | id (PK, UUID)                            |
        | entity_type (CUSTOMER/ACCOUNT/etc.)      |
        | entity_id (UUID)                         |
        | action (CREATE/UPDATE/DELETE/etc.)       |
        | performed_by (UUID, nullable - no FK)    |
        | performed_at                             |
        | old_values (JSONB)                       |
        | new_values (JSONB)                       |
        | ip_address (INET)                        |
        | user_agent                               |
        └──────────────────────────────────────────┘
```

8.2 Table Definitions

`app.users`

Column	Type	Constraints	Purpo
id	UUID	PK, default gen_random_uuid()	Unique user identifi
username	VARCHAR(100)	UNIQUE, NOT NULL	Login identifi
email	VARCHAR(255)	UNIQUE, NOT NULL	Email addres
hashed_password	VARCHAR(255)	NOT NULL	bcrypt hash o passw
role	VARCHAR(50)	NOT NULL, CHECK (ANALYST/COMPLIANCE_OFFICER/ADMIN)	RBAC role

Column	Type	Constraints	Purpo
is_active	BOOLEAN	NOT NULL, default TRUE	Soft-disable withou deletin
last_login	TIMESTAMPTZ	nullable	Track last log
created_at	TIMESTAMPTZ	NOT NULL, server_default now()	Creatic time

`app.customers`

Column	Type	Constraints
id	UUID	PK
full_name	VARCHAR(255)	NOT NULL
email	VARCHAR(255)	UNIQUE, NOT NULL
kyc_status	VARCHAR(50)	CHECK (PENDING/VERIFIED/REJECTED/UNDER_REVIEW
risk_level	VARCHAR(20)	CHECK (LOW/MEDIUM/HIGH)
risk_score	INTEGER	CHECK (0-100)
pep_flag	BOOLEAN	NOT NULL, default FALSE
sanctions_flag	BOOLEAN	NOT NULL, default FALSE
adverse_media_flag	BOOLEAN	NOT NULL, default FALSE
customer_type	VARCHAR(50)	CHECK (INDIVIDUAL/CORPORATE/FUND)
customer_metadata	JSONB	nullable

Column	Type	Constraints
country	VARCHAR(100)	nullable
residency_country	VARCHAR(100)	nullable

`app.transactions`

Column	Type	Constraints
id	UUID	PK
customer_id	UUID	FK → customers(id) CASCADE
account_id	UUID	FK → accounts(id) CASCADE
transaction_date	TIMESTAMPTZ	NOT NULL
transaction_type	VARCHAR(50)	CHECK (DEPOSIT/WITHDRAWAL/TRANSFER/FX/TRA
amount	NUMERIC(18,2)	NOT NULL, CHECK > 0
currency	VARCHAR(3)	NOT NULL, default 'USD'
risk_score	INTEGER	NOT NULL, default 0
risk_flags	JSONB	nullable
meta_country	VARCHAR(100)	nullable
meta_destination_country	VARCHAR(100)	nullable
meta_origin_country	VARCHAR(100)	nullable
meta_counterparty	VARCHAR(255)	nullable

Column	Type	Constraints
source_system	VARCHAR(100)	nullable

Indexes on transactions:

```
-- BRIN index: ~20KB for millions of rows (append-only time series)
CREATE INDEX ix_app_txn_date_brin ON app.transactions
  USING brin (transaction_date);

-- GIN index: enables @> containment queries on risk_flags JSONB
CREATE INDEX ix_app_txn_risk_flags_gin ON app.transactions
  USING gin (risk_flags);

-- B-tree indexes on foreign keys (auto-query optimization)
CREATE INDEX ON app.transactions (customer_id);
CREATE INDEX ON app.transactions (account_id);
```

`app.ml_scores`

Column	Type	Purpose
id	UUID PK	ML score record ID
transaction_id	UUID FK	Which transaction was scored
risk_scorer_version	INTEGER	Which version of RF Regressor was used
anomaly_classifier_version	INTEGER	Which version of RF Classifier was used
isolation_detector_version	INTEGER	Which version of Isolation Forest was used
is_candidate	BOOLEAN	Was a candidate model used (A/B testing)
rf_risk_score	FLOAT	Raw RF Regressor output (0-100)
anomaly_probability	FLOAT	RF Classifier probability (0.0-1.0)
isolation_score	FLOAT	Isolation Forest score mapped to 0-100
combined_score	FLOAT	Weighted ensemble score (CHECK 0-100)
anomaly_flag	BOOLEAN	True if anomaly detected
explanation	JSONB	SHAP explanation dict
features	JSONB	Full feature vector used

Why separate from transactions? The `ml_scores` table is deliberately separate from `transactions.risk_score` so the Phase 1 rules score and Phase 2 ML score never overwrite

each other. This maintains a clean audit trail and allows comparing both systems.

`app.audit_logs`

Critical design decision: `performed_by` is a `UUID` with **no foreign key** to `users`. This is intentional:

- If a user is deleted, their audit log entries must still exist for regulatory purposes
- A FK with `ON DELETE SET NULL` would work too, but the current design is simpler
- Audit logs are immutable — no UPDATE or DELETE is permitted

BRIN index on `performed_at`:

```
CREATE INDEX ix_app_audit_ts_brin ON app.audit_logs
  USING brin (performed_at);
```

This keeps the audit log index extremely compact even with millions of rows.

8.3 BRIN vs B-tree vs GIN — Explained

Index Type	Best For	Size	Lookup Speed	Notes
B-tree (default)	Random access, equality, range	Large	O(log n)	General purpose
BRIN	Append-only time-series	Tiny (~20KB)	O(n/block_range)	Only works if data is correlated with physical order
GIN	JSONB, arrays, full-text	Medium-Large	O(1) for containment	For `@>`, `?`, `@@` operators

BRIN is optimal for `transaction_date` because:

- Transactions are inserted in roughly chronological order
- BRIN stores min/max values per block range (128 pages each)
- Can be 1000x smaller than a B-tree index
- For "all transactions in July 2026" — BRIN skips all blocks outside that date range

GIN is optimal for `risk_flags` JSONB because:

- GIN inverts the JSONB keys, creating an index per key
- Enables `SELECT * FROM transactions WHERE risk_flags @> '{"triggered_rules": ["R006"]}'`

- Without GIN, this would require a sequential scan of all rows

8.4 Database Normalization

The schema follows **3rd Normal Form (3NF)**:

- **1NF:** All columns atomic (no multi-valued fields — JSONB used only for flexible metadata)
- **2NF:** All non-key attributes fully dependent on primary key
- **3NF:** No transitive dependencies

Denormalization choices:

- `transactions.risk_score` (int) — denormalized from transaction_risk_flags for fast sorting/filtering
- `customers.risk_score` and `risk_level` — cached aggregate updated by `score_customer` endpoint

8.5 SQL Queries Used in KYRO

Feature engineering — 90-day transaction history:

```
SELECT *
FROM app.transactions
WHERE customer_id = :customer_id
  AND transaction_date >= :txn_date - INTERVAL '90 days'
  AND transaction_date < :txn_date
ORDER BY transaction_date DESC;
```

Global amount statistics (cached in Redis):

```
SELECT
    AVG(amount),
    STDDEV_POP(amount),
    PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY amount ASC),
    PERCENTILE_CONT(0.9) WITHIN GROUP (ORDER BY amount ASC),
    PERCENTILE_CONT(0.99) WITHIN GROUP (ORDER BY amount ASC),
    COUNT(id)
FROM app.transactions;
```

Daily velocity check (R002):

```
SELECT COUNT(id) FROM app.transactions
WHERE customer_id = :customer_id
```

```
  AND transaction_date >= :txn_date - INTERVAL '1 day'
  AND transaction_date <= :txn_date
  AND id != :txn_id;
```

Sanctions match check (R006):
Evaluated directly in Python on `customer.sanctions_flag` — no SQL query needed.

ETL unscored transaction extraction:

```
SELECT id FROM app.transactions WHERE risk_flags IS NULL;
```

GIN containment query example (find all sanctions alerts):

```
SELECT * FROM app.transactions
WHERE risk_flags @> '{"triggered_rules": ["R006"]}';
```

8.6 Data Dictionary

Table	Approximate Row Count	Growth Rate
users	~100	Static (admin-managed)
customers	~200–100,000	Moderate (new customer onboarding)
accounts	~400–300,000	Moderate
transactions	~1M–100M+	High (daily transactions)
transaction_risk_flags	~2-5x transactions	High
alerts	~1-5% of transactions	Moderate
ml_scores	1:1 with ML-scored transactions	High
audit_logs	~3x transactions	High

8.7 CHECK Constraints — Why They Matter

CHECK constraints enforce business rules at the database level, preventing invalid data even if application code has bugs:

```
-- transaction amount must be positive
CONSTRAINT chk_transaction_amount_positive CHECK (amount > 0)

-- transaction type must be one of known values
```

```
CONSTRAINT chk_transaction_type CHECK (
    transaction_type IS NULL OR transaction_type IN
('DEPOSIT','WITHDRAWAL','TRANSFER','FX','TRADE')
)

-- ML score must be in valid range
CONSTRAINT chk_ml_score_range CHECK (combined_score >= 0 AND combined_score <=
100)

-- customer risk score must be 0-100
CONSTRAINT chk_customer_risk_score CHECK (risk_score >= 0 AND risk_score <=
100)
```

---

9. Machine Learning Documentation

9.1 ML Problem Statement

Business Problem (Simple):
Banks process millions of transactions daily. Human analysts cannot review all of them. We need a system that assigns a risk probability to each transaction automatically, flags the most suspicious ones, and explains *why* they are suspicious — in plain English that a compliance officer can use to justify a SAR filing.

ML Formulation:
This is a **hybrid supervised-unsupervised anomaly detection** problem:

- **Supervised component:** Random Forest learns from historical transactions labeled by the rules engine (proxy labels: risk_score > 50 = anomalous)
- **Unsupervised component:** Isolation Forest detects transactions that are statistically unusual without needing labels
- **Combined:** A weighted ensemble of both signals provides a robust, multi-perspective risk score

---

9.2 Dataset

Training Data Source: `app.transactions` table in PostgreSQL (default: past 365 days)

Configured by: `training_data_days: int = 365` in `config.py`

Schema of training data query:

```sql
SELECT t.*, c.*, a.*
FROM app.transactions t
JOIN app.customers c ON t.customer_id = c.id
JOIN app.accounts a ON t.account_id = a.id
WHERE t.transaction_date >= NOW() - INTERVAL '365 days'
LIMIT :limit;  -- optional
```

Minimum viable training size: The system requires > 0 transactions. In practice, meaningful ML requires at least 1,000 transactions for stable statistics.

Target Variables:

1. **RF Regressor target (y_risk):** `transaction.risk_score` — the rules engine score (0–100) used as a proxy risk label
2. **RF Classifier target (y_anomaly):** `1 if risk_score > 50 else 0` — binary anomaly label derived from the rules score threshold

Why use rules engine scores as training labels?

- The rules engine provides interpretable, domain-expert-defined labels
- ML learns to generalize rules-based patterns to detect novel variations
- This creates a "distillation" effect: ML finds subtle patterns the rules miss
- Over time, analyst feedback (`is_false_positive`) can be used for true labels

---

9.3 Feature Engineering

Total features computed: 40+

Feature computation location: `app/ml/features/engineer.py` → `compute_transaction_features()`

9.3.1 Categorical Encoding

Simple explanation: Machine learning models work with numbers, not text. So we convert text categories like "INDIVIDUAL" into numbers like 0, 1, 2.

Feature Name	Encoding
customer_type_encoded	INDIVIDUAL=0, CORPORATE=1, FUND=2
current_risk_level	LOW=0, MEDIUM=1, HIGH=2

Feature Name	Encoding
transaction_type_encoded	DEPOSIT=0, WITHDRAWAL=1, TRANSFER=2, FX=3, TRADE=4
currency_encoded	USD=0, EUR=1, GBP=2, JPY=3, CHF=4, other=0

Why label encoding, not one-hot? Random Forests handle ordinal label encoding well. For non-ordinal categoricals (like currency), one-hot would be better in production, but label encoding is a reasonable starting point.

9.3.2 Time-based Features

```
features["hour_of_day"] = float(txn_date.hour)       # 0-23
features["day_of_week"] = float(txn_date.weekday())   # 0=Monday, 6=Sunday
features["is_weekend"] = 1.0 if txn_date.weekday() >= 5 else 0.0
features["is_night"] = 1.0 if txn_date.hour < 6 else 0.0
```

Why these features matter:

- Legitimate business transactions cluster during business hours (Mon-Fri, 9am-5pm)
- Weekend/nighttime transactions are statistically unusual for corporate accounts
- Money laundering often occurs at off-hours to avoid scrutiny

9.3.3 Rolling Window Features

Simple explanation: We look at what the customer has done in the last 7, 30, and 90 days to compute "normal" behavior, then check if this transaction deviates.

```
# One SQL query fetches 90-day history
history = db.query(Transaction).filter(
    Transaction.customer_id == txn.customer_id,
    Transaction.transaction_date >= txn_date - timedelta(days=90),
    Transaction.transaction_date < txn_date
).order_by(Transaction.transaction_date.desc()).all()

# Helper: filter to window
def _window(days: int) -> list[Transaction]:
    cutoff = txn_date - timedelta(days=days)
    return [t for t in history if t.transaction_date >= cutoff]

w7, w30, w90 = _window(7), _window(30), history
```

Computed rolling features:

Feature	Description	AML Relevance
rolling_avg_7d	Mean amount in last 7 days	Baseline for amount deviation
rolling_avg_30d	Mean amount in last 30 days	Longer-term baseline
rolling_avg_90d	Mean amount in last 90 days	Broad behavioral baseline
rolling_std_7d	Std dev of 7-day amounts	Measures variability
rolling_std_30d	Std dev of 30-day amounts	Longer variability
txn_count_1h	Transaction count in last 1 hour	Rapid succession detection
txn_count_24h	Transaction count in last 24 hours	Daily velocity
txn_count_7d	Transaction count in last 7 days	Weekly frequency
txn_count_30d	Transaction count in last 30 days	Monthly frequency
amount_sum_24h	Total amount in last 24 hours	Structuring detection
amount_sum_7d	Total amount in last 7 days	Weekly exposure

9.3.4 Percentile and Z-score Features

Simple explanation: We ask "is this amount unusually large compared to all transactions in the system?"

Customer percentile:

```
customer_amounts = sorted(float(t.amount) for t in history)
rank = sum(1 for a in customer_amounts if a <= amount)
features["amount_percentile_customer"] = 100.0 * rank / len(customer_amounts)
# Example: 95.0 means this amount is larger than 95% of this customer's past
transactions
```

Global z-score:

```
global_stats = get_global_amount_stats(db)  # Cached in Redis
features["amount_zscore"] = (amount - global_stats["mean"]) /
global_stats["std"]
# Example: z-score = 3.5 means 3.5 standard deviations above global mean
```

Global percentile:

```
features["amount_percentile_global"] = _global_amount_percentile(amount,
global_stats)
```

```
# Compared to p50, p90, p99 breakpoints
```

Sample Calculation:

```
Assume global stats: mean=$5,000, std=$4,000
Transaction amount: $25,000

z-score = (25,000 - 5,000) / 4,000 = 5.0

Interpretation: This transaction is 5 standard deviations above the global
average.
This is extremely unusual - only ~0.0003% of a normal distribution falls this
far out.
The SHAP explainer would describe this as:
"Amount deviates significantly from the global baseline"
```

9.3.5 Time-since-last Features

```
if history:
    last_txn = history[0]  # Most recent (sorted desc)
    features["time_since_last_txn"] = max(0.0,
        (txn_date - last_txn.transaction_date).total_seconds() / 60.0)
    # Result in minutes

    # Time since last txn with SAME counterparty
    same_cp = next((t for t in history
                    if t.meta_counterparty == txn.meta_counterparty), None)
    features["time_since_last_txn_same_counterparty"] = \
        (txn_date - same_cp.transaction_date).total_seconds() / 60.0 if
same_cp else -1.0

    # Average time between consecutive transactions
    gaps = [(history[i].transaction_date -
history[i+1].transaction_date).total_seconds() / 60.0
            for i in range(len(history) - 1)]
    features["avg_time_between_txns"] = sum(gaps) / len(gaps) if gaps else
-1.0
```

Why -1.0 as default? When there is no history (first transaction), -1.0 is used as a sentinel value that the model learns to distinguish from real 0-minute gaps.

9.3.6 Customer-level Features

```
features.update({
    "customer_age_days": float((txn_date - customer.created_at).days),
    "account_count": float(db.query(count(Account.id)).filter(...).scalar()),
    "total_balance":
float(db.query(sum(Account.balance)).filter(...).scalar()),
    "kyc_days_since_review": float((txn_date -
customer.kyc_last_review).days),
    "pep_flag": 1.0 if customer.pep_flag else 0.0,       # CRITICAL
    "sanctions_flag": 1.0 if customer.sanctions_flag else 0.0,  # CRITICAL
    "adverse_media_flag": 1.0 if customer.adverse_media_flag else 0.0,
    "current_risk_level": float(RISK_LEVEL_ENCODING[customer.risk_level]),
    "current_risk_score": float(customer.risk_score),
    "customer_type_encoded":
float(CUSTOMER_TYPE_ENCODING[customer.customer_type]),
    "country_risk_score": country_risk_score(customer.country),       # 90.0
or 10.0
    "residency_risk_score": country_risk_score(customer.residency_country),
})
```

9.3.7 Geographic and Counterparty Features

```
HIGH_RISK_COUNTRIES = {
    "IR", "KP", "SY", "MM", "AF", "YE", "SS",
    "IRAN", "NORTH KOREA", "SYRIA", "MYANMAR", "AFGHANISTAN", "YEMEN"
}

features.update({
    "geo_diversity_score": float(len(countries_30d)),  # # unique countries in
30 days
    "unique_counterparties_7d": float(len(counterparties_7d)),
    "unique_counterparties_30d": float(len(counterparties_30d)),
    "new_counterparty_flag": 1.0 if counterparty not in known_counterparties
else 0.0,
    "counterparty_country_risk": country_risk_score(txn.meta_country),
    "destination_country_risk":
country_risk_score(txn.meta_destination_country),
    "origin_country_risk": country_risk_score(txn.meta_origin_country),
    "high_risk_country_flag": 1.0 if high_risk_hit else 0.0,
    "cross_border_flag": 1.0 if (origin_country != destination_country) else
0.0,
    "same_country_flag": 1.0 - cross_border_flag,
})
```

`country_risk_score()` function:

```python
def country_risk_score(country: str | None) -> float:
    if not country:
        return 20.0  # Unknown country — mild baseline risk
    return 90.0 if country.strip().upper() in HIGH_RISK_COUNTRIES else 10.0
```

Why 20.0 for unknown, not 0.0? An unknown country is itself a mild risk signal — legitimate transactions typically have identifiable countries.

9.3.8 Behavioral Baseline Deviation Features

Computed via `build_customer_profile()` and `calculate_deviation()`:

```python
profile = build_customer_profile(db, customer.id, as_of=txn_date)
# profile contains: avg_amount, avg_frequency, typical_hours,
typical_countries, etc.

recent_daily_count = len(w7d) / 7.0
deviations = calculate_deviation(profile, txn, recent_daily_count)
# deviations contains:
# - deviation_from_avg_amount: how much amount deviates from customer baseline
# - deviation_from_avg_frequency: how much frequency deviates
# - hour_deviation: unusual time for this customer
# - geo_deviation: unusual country for this customer
# - pattern_break_score: composite behavioral anomaly score
```

9.4 Model Selection

Model 1: Random Forest Regressor (RiskScorerModel)

What it is: An ensemble of 200 decision trees, each trained on a random subset of training data and random subset of features. Their predictions are averaged.

Configuration:

```python
RandomForestRegressor(
    n_estimators=200,      # 200 trees in the forest
    max_depth=15,          # Maximum depth of each tree
    min_samples_split=10,  # Min samples needed to split a node
    min_samples_leaf=5,    # Min samples required at leaf
    max_features="sqrt",   # Use sqrt(n_features) features per split
    bootstrap=True,        # Sample with replacement (bagging)
    random_state=42,       # Reproducibility
```

```
    n_jobs=-1,             # Use all CPU cores
)
```

Why Random Forest?

- Handles both numerical and encoded categorical features naturally
- Robust to outliers and noisy features
- Provides feature importance rankings
- Works with SHAP TreeExplainer (exact, not approximate)
- No hyperparameter tuning needed for first version (defaults are solid)

Why 200 trees? More trees = better generalization, lower variance. Diminishing returns after ~100-200. Computational cost is linear in n_estimators, but inference is fast enough.

Why max_depth=15? Prevents overfitting. Deeper trees memorize training data. 15 is a reasonable ceiling for tabular data with 40 features.

Target variable: `y_risk = transaction.risk_score` (from rules engine, 0–100)

Training process:

```
X_scaled = StandardScaler().fit_transform(X)
model.fit(X_scaled, y_risk)
```

Prediction:

```
X_scaled = scaler.transform(X)
scores = model.predict(X_scaled)
scores = np.clip(scores, 0, 100)  # Ensure 0-100 range
```

Output: Continuous float in [0, 100]

Model 2: Random Forest Classifier (AnomalyClassifier)

Configuration:

```
RandomForestClassifier(
    n_estimators=200,
    max_depth=12,           # Shallower than regressor to avoid overfitting
    min_samples_split=20,   # More conservative splitting
    class_weight="balanced", # CRITICAL: handles imbalanced classes
```

```
    random_state=42,
)
```

Why `class_weight="balanced"`?

- In real-world AML, anomalies are rare (< 5% of transactions)
- Without correction, the model learns to always predict "not anomalous" and gets 95% accuracy but misses all real fraud
- `balanced` upweights minority class samples by `n_samples / (n_classes * np.bincount(y))`

Sample calculation of class weights:

```
Training data: 1000 transactions
Anomalous (y=1): 50 samples (5%)
Normal (y=0): 950 samples (95%)

Weight for class 0 = 1000 / (2 × 950) = 0.526
Weight for class 1 = 1000 / (2 × 50) = 10.0

Effect: Each anomalous sample counts as 10 normal samples during training
```

Target variable: `y_anomaly = 1 if risk_score > 50 else 0`

Output: Probability of anomaly (0.0 to 1.0)

```
probability = model.predict_proba(X)[:, 1]  # P(class=1)
```

Model 3: Isolation Forest (UnsupervisedAnomalyDetector)

What it is: An unsupervised anomaly detection algorithm that isolates outliers by randomly partitioning the feature space. Anomalies require fewer partitions to isolate, giving them shorter "path lengths."

Configuration:

```
IsolationForest(
    n_estimators=150,      # 150 isolation trees
    contamination=0.05,    # Expected 5% anomaly rate
    max_samples="auto",    # Auto-select sample size
    random_state=42,
```

```
    n_jobs=-1,
)
```

Why Isolation Forest?

- No labels required (unsupervised)
- Detects novel anomaly types not seen during rules-engine supervised training
- Efficient: O(n log n) training, O(n log n) prediction
- `contamination=0.05` tells the model to expect 5% anomalies

How Isolation Forest works (mathematical explanation):

1. Given a dataset X, randomly select a feature and a random split value
2. Partition the dataset into two halves: values below and above the split
3. Repeat recursively until each sample is isolated
4. The number of splits needed to isolate a point = "path length"
5. **Anomalies have shorter path lengths** (easier to isolate)
6. Normal points have longer path lengths (buried deep in dense regions)

Path length formula:

```
h(x) = expected path length for point x

c(n) = 2 * H(n-1) - (2(n-1)/n)  # Expected path length for n samples
H(n) = ln(n) + 0.5772...  # Euler-Mascheroni constant ≈ 0.5772

Anomaly score: s(x, n) = 2^(-h(x)/c(n))
  s ≈ 1.0 → definitely anomalous (short path)
  s ≈ 0.5 → cannot distinguish
  s ≈ 0.0 → definitely normal (long path)
```

Sample calculation:

```
n = 1000 training samples
c(1000) = 2 × H(999) - (2×999/1000)
        = 2 × (ln(999) + 0.5772) - 1.998
        = 2 × (6.907 + 0.5772) - 1.998
        = 2 × 7.484 - 1.998
        = 14.97 - 1.998
        = 12.97

Normal transaction: h(x) = 10 steps
  s = 2^(-10/12.97) = 2^(-0.771) = 0.585 → borderline
```

```
Anomalous transaction: h(x) = 3 steps
  s = 2^(-3/12.97) = 2^(-0.231) = 0.853 → anomalous
```

sklearn's `decision_function` output:

- Returns negative values close to 0 for inliers
- Returns more negative values for outliers

KYRO's transformation:

```python
def anomaly_score(X):
    # Negate so higher = more anomalous (intuitive direction)
    return -self.model.decision_function(X_scaled)

def isolation_to_0_100(raw_score: float) -> float:
    # Map unbounded anomaly score to 0-100 scale
    return max(0.0, min(100.0, 50.0 + raw_score * 50.0))
```

Sample: raw_score = 0.6 → isolation_score = 50 + 0.6×50 = 80.0

9.5 Combined Ensemble Score Formula

Formula:

```
combined_score = (0.50 × rf_risk_score)
               + (0.35 × anomaly_probability × 100)
               + (0.15 × isolation_score)

combined_score = clamp(combined_score, 0.0, 100.0)
```

Weight rationale:
Weight	Model	Reason
0.50 (50%)	RF Regressor	Primary model; directly trained on rules-based labels
0.35 (35%)	RF Classifier	Strong anomaly signal; handles class imbalance
0.15 (15%)	Isolation Forest	Supplementary unsupervised signal

Sample Calculation:

```
Suppose:
  rf_risk_score = 75.0
  anomaly_probability = 0.85 (85% chance anomalous)
  isolation_score = 68.0

combined = (0.50 × 75.0) + (0.35 × 85.0) + (0.15 × 68.0)
         = 37.5 + 29.75 + 10.2
         = 77.45

Since 77.45 ≤ 100.0 → combined_score = 77.45
Risk level: HIGH (> 70)
Recommended action: IMMEDIATE_REVIEW
```

Anomaly flag logic:

```
anomaly_flag = (anomaly_probability > 0.5) OR (combined_score >= 71)

Examples:
  anomaly_probability=0.3, combined_score=50 → flag = False
  anomaly_probability=0.6, combined_score=40 → flag = True (prob > 0.5)
  anomaly_probability=0.4, combined_score=75 → flag = True (score >= 71)
```

9.6 SHAP Explainability

What SHAP is (simple):
Imagine a bank robber broke into a vault. To understand why the alarm failed, you ask: "What if we had removed the broken sensor? What if we had a different guard?" SHAP does this mathematically for ML models.

SHAP value definition:

```
ϕᵢ = Shapley value for feature i

ϕᵢ = Σ (|S|!(n−|S|−1)!/n!) × [f(S ∪ {i}) − f(S)]
     over all subsets S of features not containing i

where:
  n = total number of features
  S = subset of features
  f(S) = model prediction using only features in S
  f(S ∪ {i}) − f(S) = marginal contribution of feature i
```

Simple explanation of the formula:
For each possible subset of other features, we compute: "How much better does the model do when we add feature i?" We average this over all possible subsets. The result (φ_i) tells us exactly how much feature i contributed to the final prediction.

KYRO SHAP implementation:

```
class ExplainabilityEngine:
    def __init__(self, risk_scorer_model, feature_names, top_k=5):
        self._scaler = risk_scorer_model.scaler
        self.explainer = shap.TreeExplainer(risk_scorer_model.model)
        # TreeExplainer is exact (not approximate) for tree ensembles
        # Runtime: O(T × D × 2^D) where T=trees, D=max_depth

    def explain(self, features: dict[str, float]) -> dict:
        X = pd.DataFrame([features])[self.feature_names]
        X_scaled = self._scaler.transform(X)
        shap_values = self.explainer.shap_values(X_scaled)
        # shap_values[0] = per-feature SHAP values for this prediction
        # shap_values.sum() + expected_value ≈ model prediction

        pairs = sorted(zip(feature_names, shap_values[0]),
                       key=lambda p: abs(p[1]), reverse=True)
        top = pairs[:5]  # Top 5 by absolute impact
```

SHAP additivity property:

```
prediction = base_value + sum(ϕᵢ for all i)

where base_value = E[f(X)] = expected model output (average prediction)

Example:
  base_value = 35.0  (average risk score)
  ϕ_amount_zscore = +18.5  (unusually high amount)
  ϕ_high_risk_country = +12.1
  ϕ_pep_flag = +8.3
  ϕ_rolling_avg_7d = +5.8
  ϕ_weekend = +2.7
  (remaining features sum to +0.1)
  ──────────────────────────────
  prediction = 35.0 + 47.5 = 82.5
```

Human-readable descriptions:

```
_DESCRIPTIONS = {
    "amount_zscore": "Amount deviates {mag} from the global baseline",
    "high_risk_country_flag": "Transaction involves a high-risk jurisdiction",
    "pep_flag": "Customer is a Politically Exposed Person",
    "sanctions_flag": "Customer matched on a sanctions list",
    "new_counterparty_flag": "Counterparty is new to this customer",
    "pattern_break_score": "Behavior deviates {mag} from this customer's own
baseline",
}

# {mag} is "significantly" if |SHAP value| > 2 else "moderately"
# {dir} is "unusually high" if SHAP > 0 else "within normal range"
```

9.7 Model Training Pipeline

Entry: `run_training_pipeline(as_candidate, candidate_traffic_pct, limit)`

Step-by-step:

```
1. Create database session (SessionLocal())

2. train_all(db, limit):
   a. Pull transactions from last 365 days (configurable)
   b. Join with customers and accounts
   c. compute_transaction_features() for each → feature matrix X
   d. Extract y_risk = [t.risk_score for t in transactions]
   e. Extract y_anomaly = [1 if score > 50 else 0 for score in y_risk]

   f. Train RiskScorerModel:
      - X_scaled = StandardScaler().fit_transform(X)
      - model.fit(X_scaled, y_risk)

   g. Train AnomalyClassifier:
      - X_scaled = StandardScaler().fit_transform(X)
      - model.fit(X_scaled, y_anomaly)

   h. Train UnsupervisedAnomalyDetector:
      - X_scaled = StandardScaler().fit_transform(X)
      - model.fit(X_scaled)  # No labels needed!

   i. Compute metrics on training data (train set evaluation):
      Regressor: MSE, MAE, R²
      Classifier: accuracy, precision, recall, F1, AUC-ROC
```

```
   j. Return TrainingResult(risk_scorer, anomaly_classifier,
                            isolation_detector, metrics)

3. For each model:
   a. version = registry.next_version(name)  # Increment version number
   b. registry.save_model(model, name, version, metrics)
      → Pickles artifact to models/risk_scorer_v{version}.pkl

   c. If as_candidate AND active version exists:
      → registry.set_candidate(name, version, candidate_traffic_pct)
   d. Else:
      → registry.set_active(name, version)

4. Return {"versions": {model: version}, "metrics": metrics}
```

9.8 Evaluation Metrics

Regression Metrics (RiskScorerModel)

Mean Squared Error (MSE):

```
MSE = (1/n) × Σ(yᵢ − ŷᵢ)²

Variables:
  n = number of samples
  yᵢ = actual risk score (rules engine)
  ŷᵢ = predicted risk score (ML model)

Example:
  Actual: [80, 20, 55, 90, 10]
  Predicted: [75, 25, 60, 85, 15]
  Errors: [5, 5, 5, 5, 5]
  Squared: [25, 25, 25, 25, 25]
  MSE = 125/5 = 25.0

Interpretation: On average, predictions are off by √25 = 5 risk score points.
```

Mean Absolute Error (MAE):

```
MAE = (1/n) × Σ|yᵢ − ŷᵢ|
```

```
Example (same as above):
  |Errors| = [5, 5, 5, 5, 5]
  MAE = 25/5 = 5.0

Interpretation: Average absolute error is 5 risk score points.
MAE is more interpretable than MSE because it's in the same unit as y.
```

R^2 (Coefficient of Determination):

```
R² = 1 - SS_res / SS_tot

SS_res = Σ(yᵢ - ŷᵢ)²     (sum of squared residuals)
SS_tot = Σ(yᵢ - ȳ)²      (total sum of squares)
ȳ = mean(y)

Example:
  y = [80, 20, 55, 90, 10]
  ȳ = 51
  ŷ = [75, 25, 60, 85, 15]

  SS_res = 25+25+25+25+25 = 125
  SS_tot = (80-51)² + (20-51)² + (55-51)² + (90-51)² + (10-51)²
         = 841 + 961 + 16 + 1521 + 1681 = 5020

  R² = 1 - 125/5020 = 1 - 0.0249 = 0.975

Interpretation: 97.5% of variance in risk scores is explained by the ML model.
R² = 1.0 is perfect; R² = 0.0 means model is no better than predicting the
mean.
```

Classification Metrics (AnomalyClassifier)

Confusion Matrix:

```
                Predicted Normal   Predicted Anomalous
Actual Normal      TN (True Neg)    FP (False Positive)
Actual Anomalous   FN (False Neg)   TP (True Positive)
```

Precision:

```
Precision = TP / (TP + FP)

Simple: Of all transactions flagged as anomalous, what % are actually
anomalous?
```

```
Example:
  TP=45, FP=10, TN=900, FN=5
  Precision = 45 / (45+10) = 45/55 = 0.818 (81.8%)

AML Impact: Low precision → many false positives → analyst alert fatigue
```

Recall (Sensitivity):

```
Recall = TP / (TP + FN)

Simple: Of all actually anomalous transactions, what % did the model catch?

Example (same):
  Recall = 45 / (45+5) = 45/50 = 0.9 (90%)

AML Impact: Low recall → missing real money laundering → regulatory risk
```

F1 Score:

```
F1 = 2 × (Precision × Recall) / (Precision + Recall)

Simple: Harmonic mean of precision and recall.
Use when you care about both false positives AND false negatives.

Example:
  F1 = 2 × (0.818 × 0.9) / (0.818 + 0.9)
     = 2 × 0.736 / 1.718
     = 1.472 / 1.718
     = 0.857 (85.7%)
```

AUC-ROC:

```
ROC = Receiver Operating Characteristic curve
AUC = Area Under the Curve (0.0 to 1.0)

The ROC curve plots:
  X-axis: False Positive Rate = FP / (FP + TN)
  Y-axis: True Positive Rate (Recall) = TP / (TP + FN)

At different thresholds (0.0 to 1.0):
  Threshold=0.0: All flagged → TPR=1.0, FPR=1.0
  Threshold=0.5: Default → TPR=0.9, FPR=0.01
  Threshold=1.0: None flagged → TPR=0.0, FPR=0.0
```

```
AUC = 0.5 → Random guessing (worst)
AUC = 1.0 → Perfect model (ideal)
AUC = 0.95 → Very good (5% chance model ranks normal above anomalous)

Business meaning: AUC of 0.95 means that for any random pair (normal,
anomalous),
the model ranks the anomalous one higher 95% of the time.
```

9.9 Model Registry and A/B Testing

Simple explanation: When a new model is trained, we don't immediately replace the old one. Instead, we send a small percentage of traffic (e.g., 10%) to the new model and compare their performance. If the new model is better, we "promote" it.

Model registry storage:

```
models/
├── registry.json  ← Pointer file (which version is active/candidate)
├── risk_scorer_v1.pkl   ← Version 1 pickle file
├── risk_scorer_v2.pkl   ← Version 2 pickle file (candidate)
├── anomaly_classifier_v1.pkl
└── isolation_detector_v1.pkl
```

`registry.json` **structure:**

```
{
  "risk_scorer": {
    "active": 1,
    "candidate": 2,
    "candidate_traffic_pct": 10.0
  },
  "anomaly_classifier": {
    "active": 1,
    "candidate": null,
    "candidate_traffic_pct": 0
  },
  "isolation_detector": {
    "active": 1,
    "candidate": null,
    "candidate_traffic_pct": 0
```

```
  }
}
```

Traffic routing logic:

```python
def resolve_serving_version(name: str) -> tuple[int, bool]:
    entry = self.get_routing(name)
    candidate = entry.get("candidate")
    traffic_pct = entry.get("candidate_traffic_pct", 0) or 0

    if candidate is not None and random.random() * 100 < traffic_pct:
        return candidate, True   # Routed to candidate (10% chance)
    return entry["active"], False  # Routed to active (90% chance)
```

Sample A/B test flow:

```
Day 1: POST /api/v1/ml/train (as_candidate=true, candidate_traffic_pct=10)
  → Models trained, saved as v2
  → registry.json updated: candidate=2, traffic_pct=10

Day 2-7: Scoring traffic
  → 10% of requests use v2 candidate
  → 90% of requests use v1 active
  → Both store results in ml_scores with is_candidate flag

Day 7: GET /api/v1/ml/performance
  → Analysts review alerts from both versions
  → Compare false_positive_rate of candidate vs active

Decision: If candidate performs better:
  → Promote: registry.promote_candidate("risk_scorer")
    → active=2, candidate=None, traffic_pct=0
  → All traffic now uses v2
```

9.10 Automated Retraining Strategy

Weekly retraining check (Sunday 03:00 UTC):

```python
# app/services/retraining_service.py
def retrain_if_needed() -> dict:
    db = SessionLocal()
    try:
```

```
        settings = get_settings()

        # Count new transactions since last training
        # (proxy: count transactions without ml_score records)
        unscored_count = db.query(count(Transaction.id)).filter(
            ~Transaction.id.in_(
                db.query(MLScore.transaction_id)
            )
        ).scalar()

        # Retrain if enough new data
        if unscored_count >= settings.retrain_threshold:  # default: 1000
            return run_training_pipeline(as_candidate=True,
candidate_traffic_pct=10.0)

        return {"status": "skipped", "reason": f"Only {unscored_count} new
samples"}
    finally:
        db.close()
```

Retraining trigger conditions:

- 1,000+ new transactions since last training (configurable: `retrain_threshold`)
- Always deployed as candidate (10% traffic) for safety
- Analyst can promote manually via `ModelRegistry.promote_candidate()`

Why not retrain daily?

- Random Forest training is expensive (minutes for large datasets)
- New data needs time to accumulate for meaningful model updates
- Weekly is a good balance between freshness and stability

9.11 Model Limitations

Limitation	Description	Mitigation
Proxy labels	Trained on rules engine output, not true fraud labels	Collect analyst feedback for true labels in v2
Class imbalance	Rare anomalies; `class_weight="balanced"` partially addresses	SMOTE oversampling in future

Limitation	Description	Mitigation
Feature staleness	Global stats cached for 1 hour	Reduce TTL in high-volume environments
No drift detection	No automated alert when data distribution changes	Add population stability index (PSI) monitoring
Pickle security	Model files are not encrypted	Use secure storage with access controls in production
Cold start	Requires minimum training data; returns 409 before first train	Seed initial training data before production launch
Single-region	No distributed training	Spark MLlib for multi-node training at scale

---

9.12 Model Monitoring Plan

Metric	Frequency	Alert Threshold
False positive rate	Daily	> 30%
Precision	Weekly	< 70%
Combined score distribution	Weekly	Mean drift > 10 points
Feature distribution PSI	Weekly	PSI > 0.2
Retraining trigger	Weekly	1,000+ new transactions
Model latency	Real-time	P95 > 200ms

---

10. AI Workflow

> **Note:** The current KYRO system does not use LLMs, RAG, embeddings, or vector databases. The "AI" in KYRO refers to the ML ensemble engine described in Section 9.

10.1 Current AI Components

Component	Technology	Purpose
Risk Scoring AI	Random Forest Regressor (sklearn)	Continuous risk score

Component	Technology	Purpose
Anomaly Detection AI	Random Forest Classifier (sklearn)	Binary anomaly classification
Outlier Detection AI	Isolation Forest (sklearn)	Unsupervised novelty detection
Explainability AI	SHAP TreeExplainer	Feature attribution
Decision Support	Rules Engine (deterministic)	Regulatory rule enforcement

10.2 Future AI Enhancements (Planned)

LLM Integration for SAR Drafting

Plan: Use OpenAI GPT-4 or Claude to automatically draft Suspicious Activity Reports based on:

- Transaction data
- SHAP explanation
- Customer KYC profile
- Triggered rules

Planned prompt structure:

```
System: You are an AML compliance expert drafting a Suspicious Activity
Report.

Context:
  Customer: {customer.full_name}, type={customer.customer_type}
  Transaction: ${amount} {currency} on {date}
  Risk Score: {combined_score}/100
  Triggered Rules: {triggered_rules}
  ML Explanation: {shap_summary}

Task: Draft a SAR narrative paragraph of 200-300 words that:
1. Describes the suspicious activity
2. References specific risk factors
3. Uses language appropriate for FinCEN filing
```

Vector Database for Similar Case Retrieval

Plan: Store SHAP explanation vectors in Pinecone or pgvector, enabling:

- "Find 5 most similar historical alerts"

- Analyst guidance: "Similar cases were investigated and resolved as..."

Technology stack: `pgvector` extension (PostgreSQL native), or Pinecone for scale

Graph Network Analysis

Plan: Build customer relationship graphs to detect:

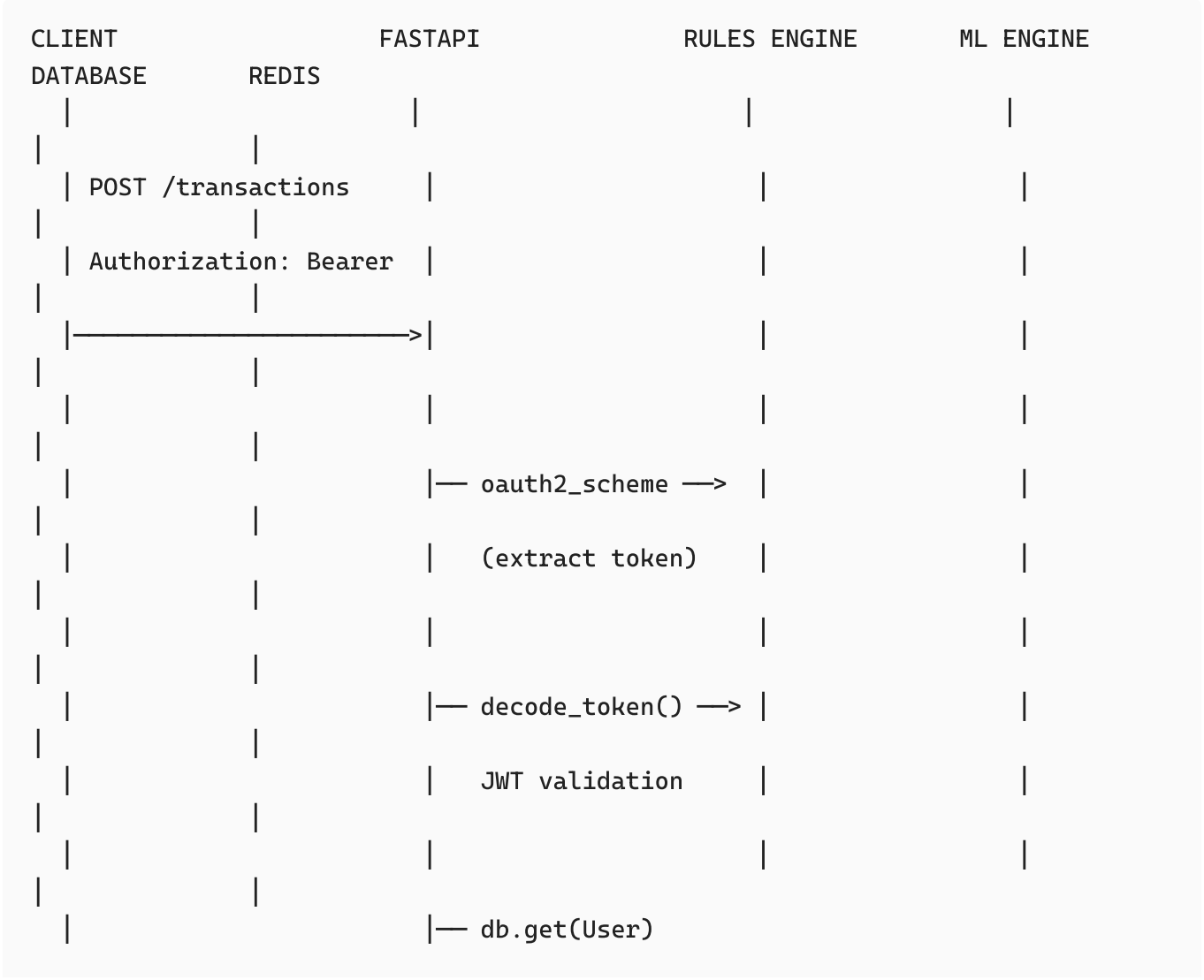
- Money mule networks (hub-and-spoke patterns)
- Layering through multiple accounts
- Shell company networks

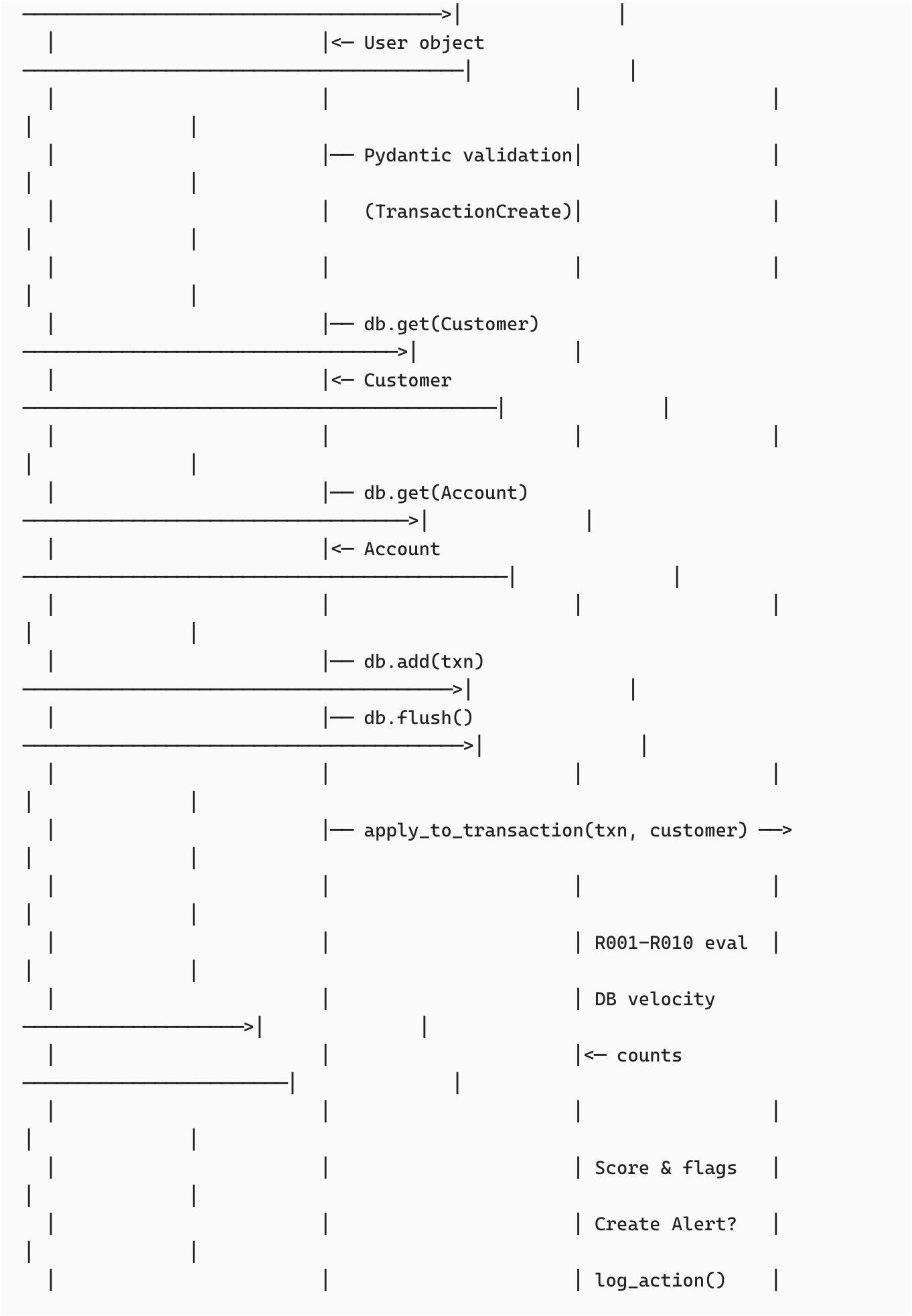
Technology: NetworkX for initial implementation, Neo4j for production

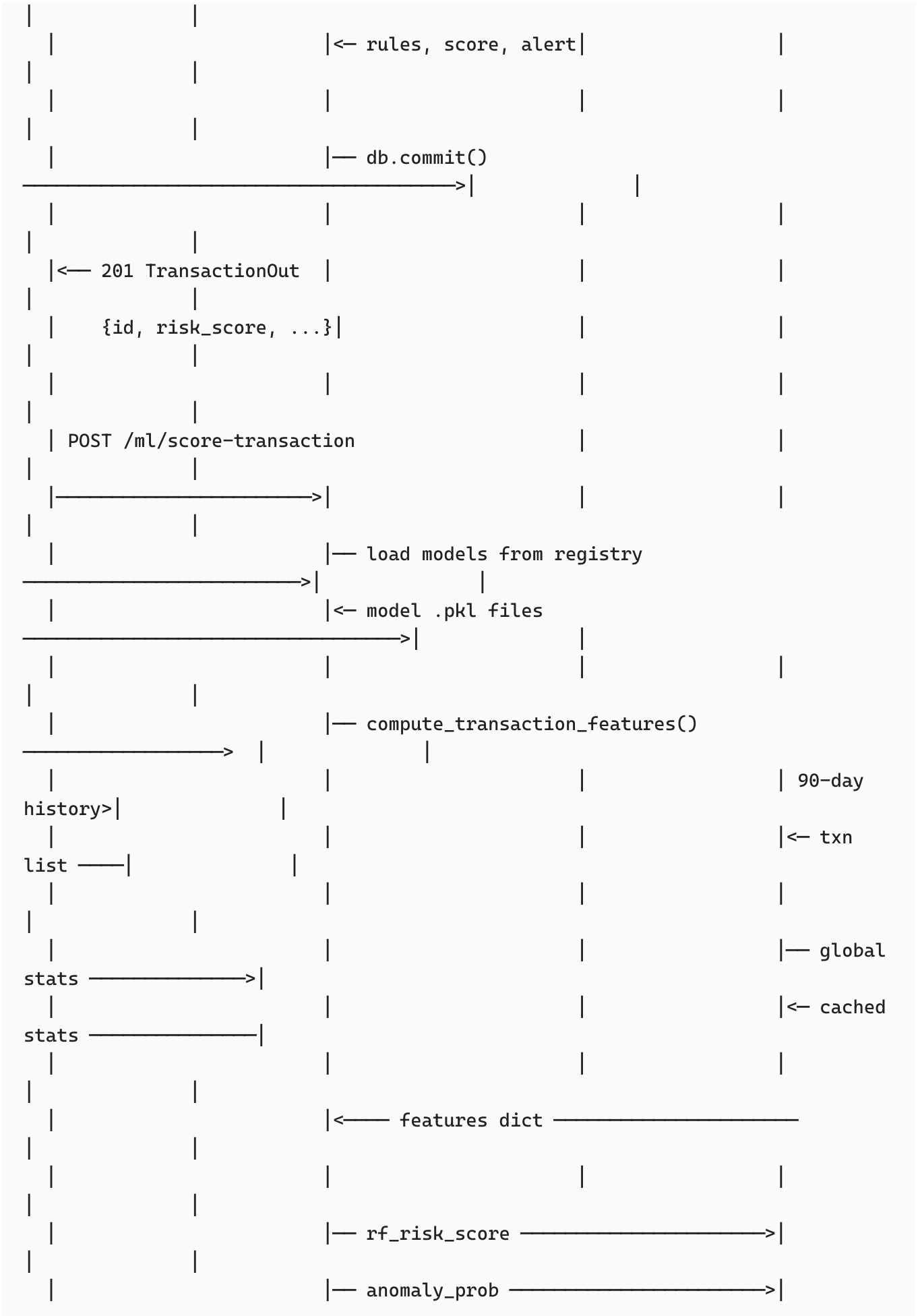
---

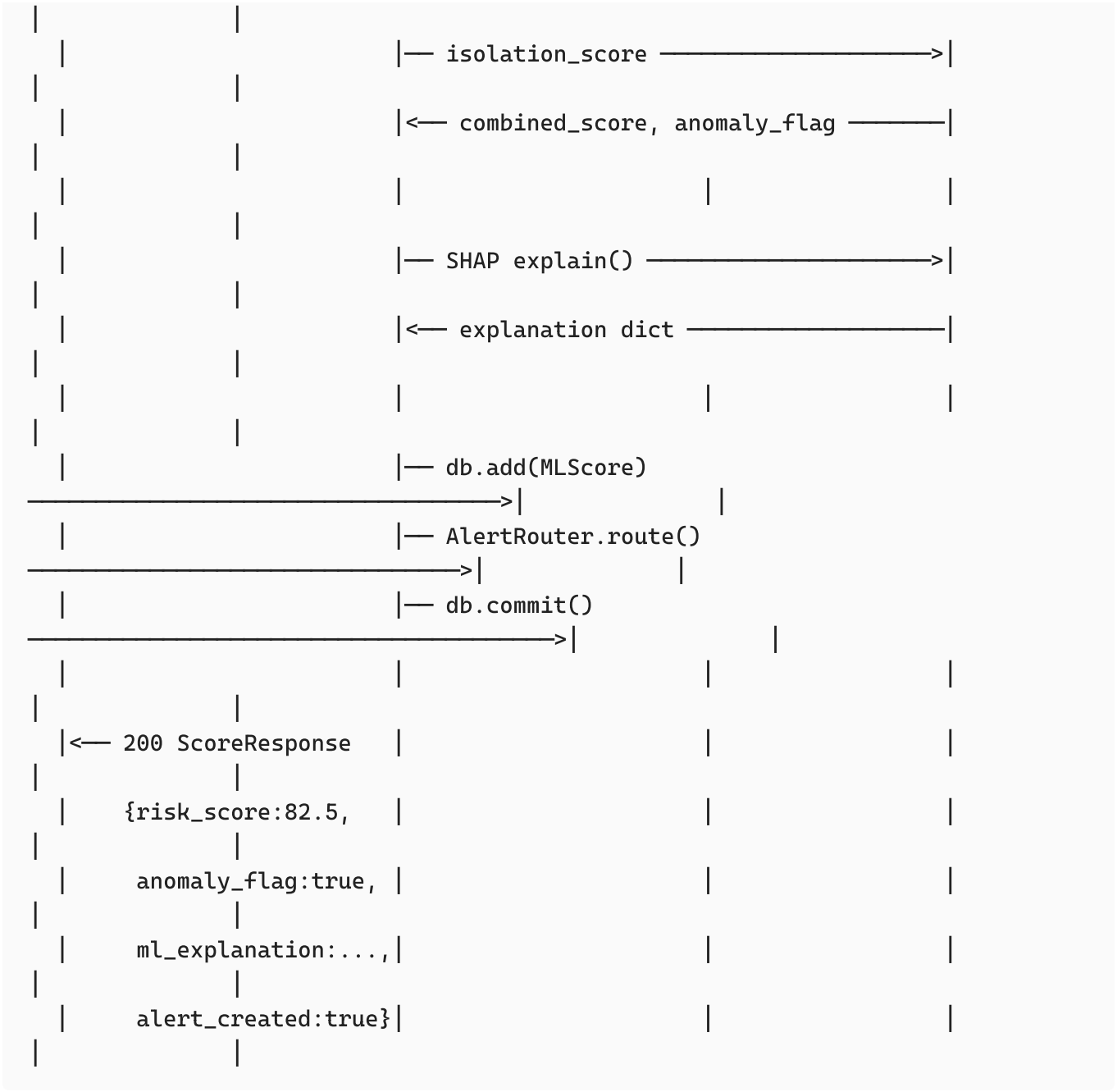
11. API Flow

11.1 Complete Request Sequence Diagram

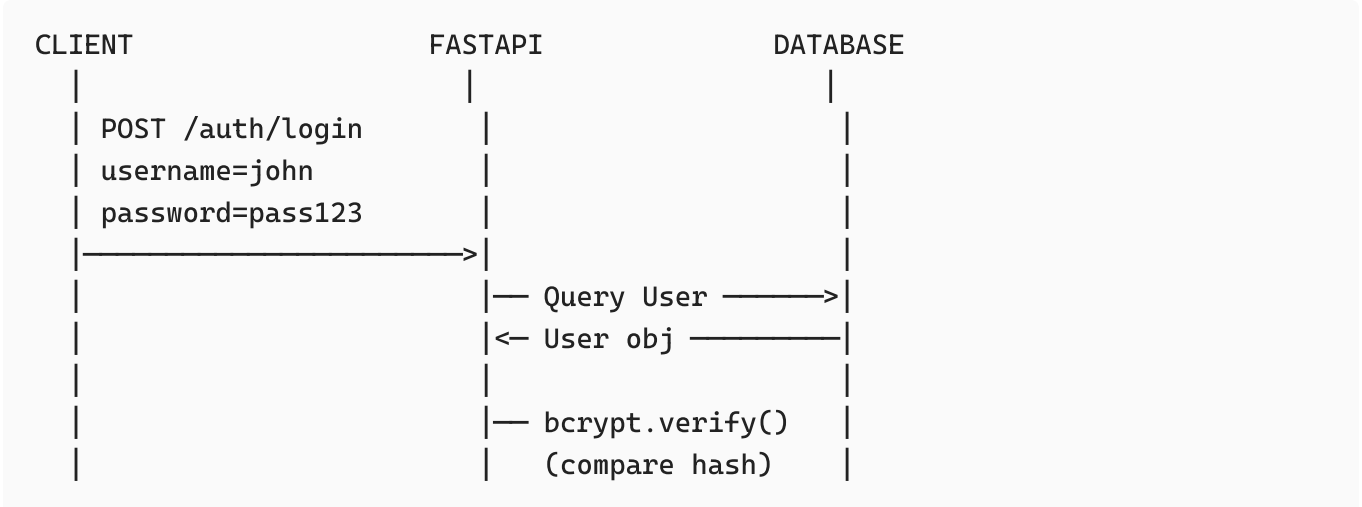



|<- User object
|-- Pydantic validation
(TransactionCreate)
|-- db.get(Customer)
|<- Customer
|-- db.get(Account)
|<- Account
|-- db.add(txn)
|-- db.flush()
|-- apply_to_transaction(txn, customer) -->
R001-R010 eval
DB velocity
|<- counts
Score & flags
Create Alert?
log_action()


|<- rules, score, alert|
|-- db.commit()
|<-- 201 TransactionOut
{id, risk_score, ...}
POST /ml/score-transaction
|-- load models from registry
|<- model .pkl files
|-- compute_transaction_features()
90-day history>|
|<- txn list ----|
|-- global stats ------------>|
|<- cached stats -------------|
|<---- features dict ----------
|-- rf_risk_score ------------------------>|
|-- anomaly_prob ------------------------->|



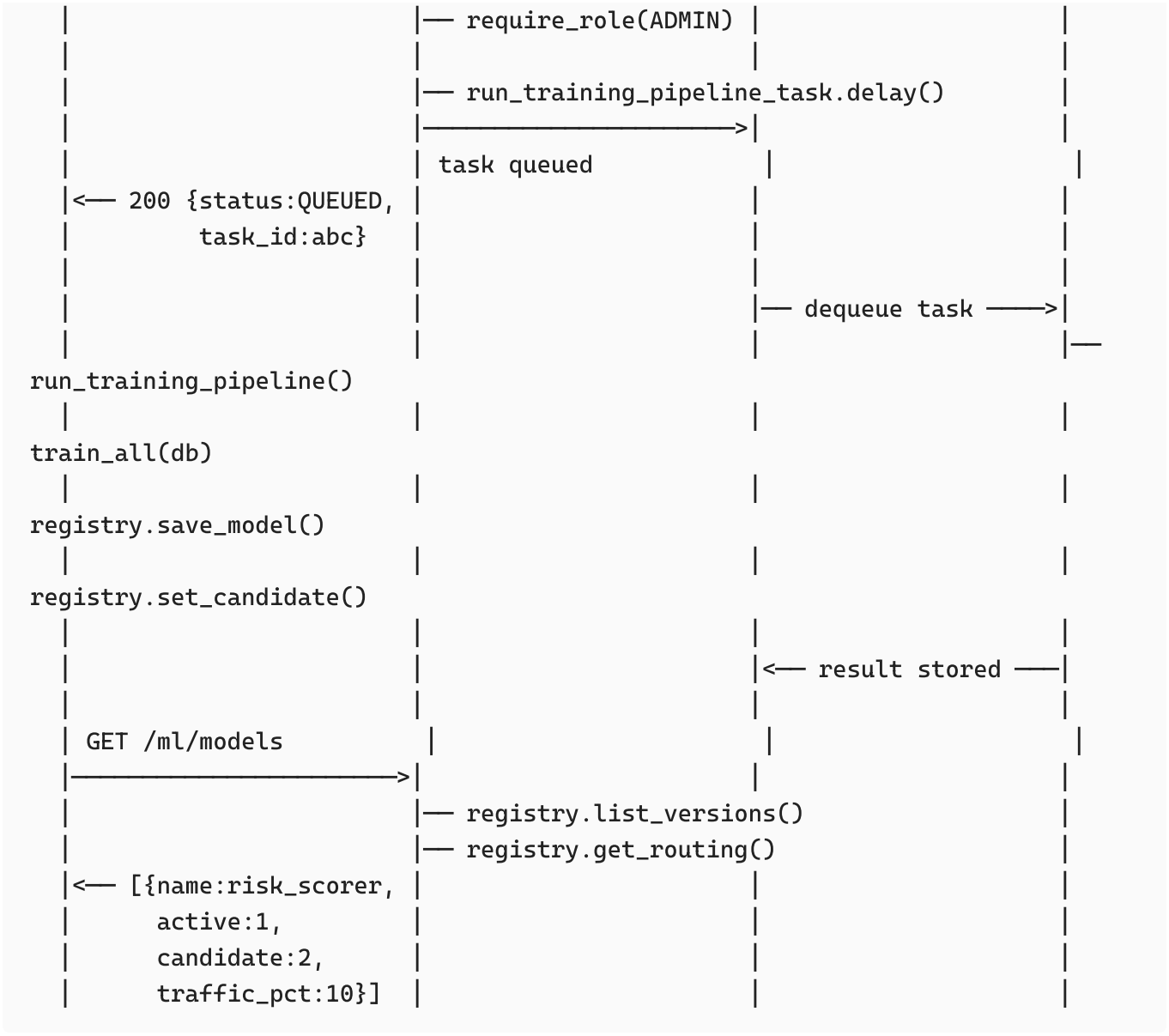
11.2 Authentication Flow Sequence



```
   |                          |                          |
   |                          |-- user.last_login=now|
   |                          |-- db.commit() ------>|
   |                          |                          |
   |                          |-- create_access_token(user.id, role)
   |                          |   JWT payload:       |
   |                          |   {sub:id, role:...,|
   |                          |    type:access,      |
   |                          |    exp:now+30min}    |
   |                          |   Sign with HS256    |
   |                          |                          |
   |                          |-- create_refresh_token(user.id, role)
   |                          |   JWT payload:       |
   |                          |   {sub:id, role:...,|
   |                          |    type:refresh,     |
   |                          |    exp:now+7days}    |
   |                          |                          |
   |<-- 200 Token             |                          |
   |    {access_token:...,    |                          |
   |     refresh_token:...,   |                          |
   |     token_type:bearer}   |                          |
   |                          |                          |
   | [30 minutes later]       |                          |
   | POST /auth/refresh       |                          |
   | {refresh_token: "..."}   |                          |
   |------------------------->|                          |
   |                          |-- decode_token()     |
   |                          |   verify signature   |
   |                          |   check exp          |
   |                          |   check type=refresh|
   |                          |-- db.get(User) ----->|
   |                          |<- User --------------|
   |                          |-- new tokens ------->|
   |<-- 200 Token (new)       |                          |
```

11.3 ML Training Async Flow

```
ADMIN CLIENT            FASTAPI              REDIS (Celery)       CELERY
WORKER
   |                        |                        |                    |
   | POST /ml/train         |                        |                    |
   | {run_async: true,      |                        |                    |
   |  as_candidate: true,   |                        |                    |
   |  candidate_pct: 10}    |                        |                    |
   |------------------------>|                        |                    |
```



---

12. Authentication & Authorization

12.1 JWT Token Architecture

What JWT is: A compact, URL-safe way to encode claims between two parties. It consists of three Base64Url-encoded parts: Header.Payload.Signature

JWT Structure in KYRO:

```
Header:
{
  "alg": "HS256",  // HMAC with SHA-256
  "typ": "JWT"
}
```

```
Payload (Access Token):
{
  "sub": "550e8400-e29b-41d4-a716-446655440000",  // User UUID
  "role": "ANALYST",                               // RBAC role
  "type": "access",                                // Token type
  "iat": 1722320400,                               // Issued at (Unix
timestamp)
  "exp": 1722322200                                // Expires at (iat + 30
minutes)
}

Signature:
HMACSHA256(
  base64url(header) + "." + base64url(payload),
  SECRET_KEY  // From .env file
)
```

Why HS256? Symmetric algorithm — same key for signing and verification. Simpler for single-service deployments. For multi-service architectures, RS256 (asymmetric RSA) is preferable.

12.2 Token Lifecycle

Token Type	Expiry	Purpose
Access Token	30 minutes	API authentication (short-lived for security)
Refresh Token	7 days	Obtain new access tokens without re-login

Security rationale for short access token expiry:

- If an access token is stolen, it becomes invalid within 30 minutes
- The refresh token allows seamless renewal without user friction
- Refresh tokens should be stored in httpOnly cookies (not localStorage) in a web frontend

12.3 RBAC (Role-Based Access Control)

Three roles in KYRO:

Role	Permissions
ANALYST	Read all endpoints; no training or admin actions
COMPLIANCE_OFFICER	All ANALYST permissions + resolve alerts + KYC reviews
ADMIN	All permissions + train models + manage users

Implementation:

```
def require_role(*roles: str):
    def _check(user: User = Depends(get_current_user)) -> User:
        if user.role not in roles:
            raise HTTPException(status.HTTP_403_FORBIDDEN,
                                "Insufficient permissions")
        return user
    return _check

# Usage in router:
@router.post("/train")
def train(data: TrainRequest,
          user: User = Depends(require_role("ADMIN"))):  # Only ADMIN
    ...
```

Protected endpoints by role:

Endpoint	Required Role
POST /auth/register	None (public)
POST /auth/login	None (public)
GET /api/v1/health	None (public)
All other GET endpoints	ANALYST or above
POST /api/v1/transactions	ANALYST or above
PATCH /api/v1/alerts/{id}	ANALYST or above
POST /api/v1/ml/train	ADMIN only

12.4 Password Security

bcrypt hashing:

```
_pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")

# Storing password (registration):
hashed = _pwd_context.hash("my_password")
# Result: $2b$12$N9qo8uLOickgx2ZMRZoMyeIjZAgcfl7p92ldGxad68LJZdL17lhWy
# The $12$ means cost factor 12 = 2^12 = 4096 bcrypt iterations

# Verifying password (login):
```

```
valid = _pwd_context.verify("my_password", hashed)  # True
valid = _pwd_context.verify("wrong_pass", hashed)   # False
```

Why bcrypt over SHA256?

- bcrypt is intentionally slow (cost factor = 12 by default = 4096 iterations)
- SHA256 can compute billions of hashes per second on a GPU → brute-force feasible
- bcrypt's adaptive cost factor can be increased as hardware gets faster
- bcrypt automatically includes a random salt (embedded in output)

bcrypt cost factor analysis:

```
Cost factor 12 → 4096 iterations
Time per hash (modern CPU): ~100ms
Time for 1 million guesses: 100,000 seconds (~28 hours)
→ Brute-force of common passwords is infeasible
```

12.5 Stateless JWT Design

The current implementation uses **stateless JWT**:

- No server-side session storage
- Token validity determined entirely by signature and expiry
- **Implication:** Logout does not invalidate tokens server-side

Current logout:

```
@router.post("/logout")
def logout(_: User = Depends(get_current_user)) -> dict:
    # Stateless JWT: client discards tokens.
    # Server-side revocation is a hardening item for a later phase.
    return {"detail": "Logged out"}
```

Future improvement: Redis token denylist

```
# On logout:
redis_client.setex(f"denylist:{token_jti}", expiry_seconds, "1")

# On each request:
if redis_client.exists(f"denylist:{claims['jti']}"):
    raise HTTPException(401, "Token revoked")
```

---

13. Security

13.1 OWASP Top 10 Coverage

OWASP Risk	Status	KYRO Implementation
A01: Broken Access Control	✅ Addressed	RBAC with `require_role()`
A02: Cryptographic Failures	✅ Addressed	bcrypt passwords, HS256 JWT
A03: Injection	✅ Addressed	SQLAlchemy ORM (parameterized queries)
A04: Insecure Design	✅ Addressed	Least-privilege registration (always ANALYST)
A05: Security Misconfiguration	⚠️ Partial	`allow_origins=["*"]` needs restriction in prod
A06: Vulnerable Components	⚠️ Ongoing	Pin dependency versions; run `pip audit`
A07: Auth Failures	✅ Addressed	JWT + bcrypt; inactive user check
A08: Data Integrity Failures	✅ Addressed	Pydantic validation + CHECK constraints
A09: Logging Failures	✅ Addressed	Audit log for all mutations
A10: SSRF	N/A	No outbound HTTP requests in current version

13.2 SQL Injection Prevention

Why KYRO is protected:
SQLAlchemy ORM generates parameterized queries automatically. User input **never** concatenates into SQL strings.

Safe (SQLAlchemy):

```
# This generates: SELECT * FROM users WHERE username = $1
# The $1 is a parameterized placeholder — user input goes into parameter, not SQL
user = db.query(User).filter(User.username == data.username).first()
```

Dangerous (raw SQL — NOT used in KYRO):

```
# NEVER do this:
db.execute(f"SELECT * FROM users WHERE username = '{data.username}'")
# Injection: username = "'; DROP TABLE users; --"
```

13.3 Input Validation

Every API request body is validated by Pydantic schemas before reaching business logic:

```
class TransactionCreate(BaseModel):
    customer_id: uuid.UUID           # Must be valid UUID
    account_id: uuid.UUID            # Must be valid UUID
    amount: float = Field(gt=0)      # Must be > 0
    transaction_date: datetime       # Must be valid ISO datetime
    currency: str = Field(max_length=3)  # Max 3 chars (ISO currency)
    transaction_type: Optional[str]  # Optional
```

If validation fails:

```
{
  "detail": [
    {
      "type": "greater_than",
      "loc": ["body", "amount"],
      "msg": "Input should be greater than 0",
      "input": -500.0
    }
  ]
}
```

HTTP 422 is returned automatically — route handler never executes.

13.4 Secret Management

Environment variables used for all secrets:

```
SECRET_KEY=change-me-in-production  # JWT signing key
DB_PASSWORD=kyro_pass               # Database password
PGADMIN_PASSWORD=admin123           # pgAdmin UI password
```

Production requirements:

1. Generate with `openssl rand -hex 32` (64 hex chars = 256 bits)
2. Store in HashiCorp Vault, AWS Secrets Manager, or Azure Key Vault
3. Inject as environment variables at runtime (never in code or git)
4. Rotate every 90 days

Why `.env.example` is committed but `.env` is not:

- `.env.example` shows the *structure* of required variables with dummy values
- `.gitignore` includes `.env` to prevent accidental credential commits
- Real `.env` is created from `.env.example` by each developer/deployment

13.5 Rate Limiting (Future Hardening)

Current status: No rate limiting implemented.

Recommended implementation:

```
# Using slowapi (Starlette-compatible rate limiter)
from slowapi import Limiter
from slowapi.util import get_remote_address

limiter = Limiter(key_func=get_remote_address)

@router.post("/auth/login")
@limiter.limit("5/minute")  # 5 login attempts per minute per IP
def login(...):
    ...
```

Critical endpoints to rate-limit:

- POST /auth/login — Prevent brute force (5/minute/IP)
- POST /auth/register — Prevent mass registration (10/hour/IP)
- POST /ml/train — Expensive operation (3/hour for ADMIN)

13.6 CORS Security

Current (Development):

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],       # Allows ANY domain - OK for dev only
    allow_credentials=True,
    allow_methods=["*"],
```

```
    allow_headers=["*"],
)
```

Production recommendation:

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=[
        "https://kyro.yourcompany.com",    # Production frontend
        "https://staging.kyro.yourcompany.com"  # Staging
    ],
    allow_credentials=True,
    allow_methods=["GET", "POST", "PATCH", "DELETE"],
    allow_headers=["Authorization", "Content-Type"],
)
```

13.7 Database Security

Separation of concerns:

- The API application uses `kyro_user` with limited permissions
- Schema `app` is owned by this user
- pgAdmin runs on a separate container and is never exposed in production

Security roles in `pipeline/migrations/004_security_roles.sql`:

```
-- Read-only role for analytics/reporting
CREATE ROLE kyro_readonly;
GRANT SELECT ON ALL TABLES IN SCHEMA app TO kyro_readonly;

-- Application role (DML but no DDL)
CREATE ROLE kyro_app;
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA app TO kyro_app;
```

13.8 Audit Logging for Compliance

Every data mutation triggers an audit log entry:

```
def log_action(db, entity_type, entity_id, action, performed_by,
               old_values=None, new_values=None,
               ip_address=None, user_agent=None):
    db.add(AuditLog(
        entity_type=entity_type,
        entity_id=entity_id,
```

```
        action=action,
        performed_by=performed_by,  # User UUID (no FK — intentional)
        old_values=old_values,
        new_values=new_values,
        ip_address=ip_address,
        user_agent=user_agent,
    ))
```

Immutability guarantee:

- No UPDATE or DELETE operations are ever run on `audit_logs`
- The table is append-only by application convention
- In production, a PostgreSQL row-level security policy can enforce this at DB level:

```
CREATE POLICY audit_log_insert_only ON app.audit_logs
  FOR INSERT WITH CHECK (true);
ALTER TABLE app.audit_logs ENABLE ROW LEVEL SECURITY;
```

---

14. Docker & Containerization

14.1 Docker Compose Architecture

KYRO uses 7 Docker services that communicate over a shared bridge network `kyro_net`.

Full service dependency graph:

```
postgres (healthcheck: pg_isready)
    depends_on (healthy) by:
    - pgadmin
    - pipeline (restart: "no" runs once then exits)
    - api
    - celery_worker
    - celery_beat

redis (healthcheck: redis-cli ping)
    depends_on (healthy) by:
    - api
    - celery_worker
    - celery_beat
```

14.2 Service Definitions

PostgreSQL Service

```
postgres:
  image: postgres:16-alpine         # Lightweight Alpine image (~80MB)
  container_name: kyro_postgres
  restart: unless-stopped           # Auto-restart on failure
  environment:
    POSTGRES_DB: ${DB_NAME:-kyro_aml}
    POSTGRES_USER: ${DB_USER:-kyro_user}
    POSTGRES_PASSWORD: ${DB_PASSWORD:-kyro_pass}
    PGDATA: /var/lib/postgresql/data/pgdata
  ports:
    - "5434:5432"     # Host 5434 to Container 5432 (avoids local conflicts)
  volumes:
    - postgres_data:/var/lib/postgresql/data   # Persistent named volume
    # SQL migrations run automatically on first startup:
    - ./pipeline/migrations/001_create_schemas_tables.sql:/docker-entrypoint-
initdb.d/01_tables.sql
    - ./pipeline/migrations/002_create_indexes.sql:/docker-entrypoint-
initdb.d/02_indexes.sql
    - ./pipeline/migrations/003_triggers_procedures_views.sql:/docker-
entrypoint-initdb.d/03_triggers.sql
    - ./pipeline/migrations/004_security_roles.sql:/docker-entrypoint-
initdb.d/04_security.sql
  command: >
    postgres
      -c config_file=/etc/postgresql/postgresql.conf
      -c log_statement=ddl
      -c log_min_duration_statement=500
      -c shared_preload_libraries=pg_stat_statements
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U ${DB_USER:-kyro_user} -d ${DB_NAME:-
kyro_aml}"]
    interval: 10s
    timeout: 5s
    retries: 5
```

Why docker-entrypoint-initdb.d/? PostgreSQL's Docker image automatically runs `.sql` files in this directory on **first startup only** (when the data directory is empty). This creates the schema, indexes, triggers, and security roles automatically.

API Service (FastAPI)

```
api:
  build:
```

```
    context: .
    dockerfile: docker/Dockerfile.api
  container_name: kyro_api
  environment:
    DATABASE_URL: postgresql+psycopg://...@postgres:5432/kyro_aml
    # Uses 'postgres' hostname — Docker DNS resolves service names
    REDIS_URL: redis://redis:6379/0
    SECRET_KEY: ${SECRET_KEY:-change-me-in-production}
  ports:
    - "${API_HOST_PORT:-8000}:8000"
  volumes:
    - ./models:/app/models    # Bind mount: models shared between host and
container
  depends_on:
    postgres:
      condition: service_healthy
    redis:
      condition: service_healthy
```

Celery Worker and Beat

```
celery_worker:
  build:
    context: .
    dockerfile: docker/Dockerfile.api   # SAME image as api
  command: celery -A app.tasks.celery_app worker --loglevel=info
  healthcheck:
    test: ["CMD-SHELL", "celery -A app.tasks.celery_app inspect ping -d
celery@$$HOSTNAME"]
    interval: 30s
    retries: 3

celery_beat:
  command: celery -A app.tasks.celery_app beat --loglevel=info
  healthcheck:
    disable: true    # beat does not respond to ping like a worker
```

14.3 Named Volumes vs Bind Mounts

Mount Type	Used For	Behavior
Named volume (postgres_data)	PostgreSQL data	Docker-managed, persists across recreations

Mount Type	Used For	Behavior
Named volume (redis_data)	Redis persistence	Survives container restart
Bind mount (./models:/app/models)	ML model .pkl files	Host and container share same directory
Bind mount (./pipeline:/app/pipeline)	Pipeline code	Live code changes visible in container

14.4 Docker Network

```
networks:
  kyro_net:
    driver: bridge
```

All 7 services share one virtual bridge network. They communicate by service name (Docker internal DNS). The network is isolated from other Docker networks.

---

15. Deployment

15.1 Environments

Environment	Purpose	Configuration
Development	Local developer machine	docker compose up with default .env
Testing	Automated CI testing	pytest with SQLite in-memory DB
UAT	Business validation	Staging server with real data subset
Production	Live system	Cloud deployment with managed services

15.2 Local Development Setup

```
# Step 1: Clone the repository
git clone https://github.com/yourorg/kyro.git
cd kyro/KYRO_NEW

# Step 2: Copy environment template and set SECRET_KEY
cp .env.example .env
# Edit .env: SECRET_KEY=$(openssl rand -hex 32)
```

```
# Step 3: Start all services
docker compose up -d

# Step 4: Verify health
docker compose ps
curl http://localhost:8000/api/v1/health
# Expected: {"status": "ok", "service": "KYRO Risk Assessment"}

# Step 5: Open Swagger UI
# http://localhost:8000/docs

# Step 6: Register user, then login for Bearer token, then train models
```

15.3 Production Deployment Checklist

- [ ] SECRET_KEY = cryptographically random 256-bit value (openssl rand -hex 32)
- [ ] DEBUG=False
- [ ] Restrict CORS allow_origins to actual frontend domain
- [ ] Use managed PostgreSQL (AWS RDS, Azure Database) - not Docker
- [ ] Use managed Redis (AWS ElastiCache, Azure Cache for Redis)
- [ ] Enable TLS/HTTPS with valid certificate (Nginx reverse proxy or cloud LB)
- [ ] Set DB_SSL_MODE=verify-full for encrypted DB connection
- [ ] Configure log shipping (CloudWatch, ELK stack)
- [ ] Set up health check monitoring (Prometheus/Grafana)
- [ ] Enable PostgreSQL connection pooling (PgBouncer)
- [ ] Run `pip audit` to check for known vulnerabilities
- [ ] Pin all dependency versions (use == not >= in requirements files)
- [ ] Remove pgAdmin from production Docker Compose

15.4 Rollback Strategy

```
# Rollback API to previous container version
docker pull registry.example.com/kyro-api:previous-sha
docker compose up -d --no-deps api

# Rollback ML model (edit registry.json)
# Change: {"risk_scorer": {"active": 2}} back to {"active": 1}
# No restart needed - registry is read on each scoring request
```

---

16. Logging

16.1 Application Logs

FastAPI/Uvicorn generates access logs automatically:

```
INFO:     172.18.0.1:52340 - "POST /api/v1/transactions HTTP/1.1" 201 Created
INFO:     172.18.0.1:52341 - "POST /api/v1/ml/score-transaction HTTP/1.1" 200
OK
INFO:     172.18.0.1:52342 - "POST /api/v1/auth/login HTTP/1.1" 401
Unauthorized
```

Log level set via: `LOG_LEVEL=DEBUG|INFO|WARNING|ERROR` in .env

16.2 Audit Logs

Every data mutation is recorded in `app.audit_logs`:

```sql
SELECT * FROM app.audit_logs
WHERE entity_type = 'TRANSACTION'
  AND action = 'CREATE'
  AND performed_at >= NOW() - INTERVAL '1 day'
ORDER BY performed_at DESC;
```

Sample audit log record:

```json
{
  "id": "uuid...",
  "entity_type": "TRANSACTION",
  "entity_id": "txn-uuid...",
  "action": "CREATE",
  "performed_by": "user-uuid...",
  "performed_at": "2026-07-30T12:05:33.123Z",
  "old_values": null,
  "new_values": {"amount": 15000.00, "transaction_type": "TRANSFER"},
  "ip_address": "10.0.0.5"
}
```

16.3 ML Logs

Every ML scoring result is persisted in `app.ml_scores`:

```sql
SELECT ms.transaction_id, ms.combined_score, ms.anomaly_flag,
       ms.risk_scorer_version, ms.is_candidate, ms.created_at
FROM app.ml_scores ms
ORDER BY ms.created_at DESC
LIMIT 100;
```

The `is_candidate` flag enables A/B test analysis by comparing candidate vs active model performance.

16.4 Celery Task Logs

```bash
# Follow Celery worker logs
docker logs kyro_celery_worker --follow

# Sample output:
# [02:00:00] INFO run_daily_etl_pipeline received
# [02:00:01] INFO Extracted 47 unscored transactions
# [02:00:02] INFO Scored 47 transactions. Result: {"scored": 47}
# [02:00:02] INFO Task completed successfully
```

16.5 PostgreSQL Slow Query Logging

Configured in docker-compose.yml:

```
-c log_min_duration_statement=500   # Log queries > 500ms
-c log_statement=ddl                # Log all DDL operations
```

Access logs:

```bash
docker logs kyro_postgres 2>&1 | grep "duration:"
```

---

17. Performance Optimization

17.1 Database Optimization

BRIN Index for Time-Series

```sql
-- BRIN index: ~20KB for 1M rows vs ~50MB for B-tree
CREATE INDEX ix_app_txn_date_brin ON app.transactions
USING brin (transaction_date);
```

```sql
-- Efficient date range query:
SELECT * FROM app.transactions
WHERE transaction_date BETWEEN '2026-07-01' AND '2026-07-31';
-- PostgreSQL skips all data blocks outside this range
```

BRIN is optimal because transactions are inserted in chronological order — the physical file order matches the date order.

GIN Index for JSONB

```sql
-- Containment query on risk_flags
SELECT * FROM app.transactions
WHERE risk_flags @> '{"triggered_rules": ["R006"]}';

-- Without GIN: full table scan (slow)
-- With GIN index: direct lookup (fast)
```

Connection Pooling

```python
engine = create_engine(
    settings.database_url,
    pool_size=10,        # 10 persistent connections
    max_overflow=20,     # 20 burst connections
    pool_pre_ping=True   # Health check before use
)
```

Single History Query for Feature Engineering

```python
# One SQL query fetches all 90-day history (not 3 separate queries)
history = db.query(Transaction).filter(
    Transaction.customer_id == txn.customer_id,
    Transaction.transaction_date >= txn_date - timedelta(days=90),
    Transaction.transaction_date < txn_date,
).all()

# Then filter in Python for each window:
w7 = [t for t in history if t.transaction_date >= txn_date -
timedelta(days=7)]
w30 = [t for t in history if t.transaction_date >= txn_date -
timedelta(days=30)]
```

17.2 Redis Caching

```
GLOBAL_STATS_CACHE_KEY = "ml:global_amount_stats"
GLOBAL_STATS_TTL_SECONDS = 3600  # 1 hour TTL

# Cache hit saves 1 PostgreSQL percentile query per request
# For 1000 requests/hour: 999 cache hits, 1 cache miss (DB query)
```

17.3 Pagination

All list endpoints return paginated responses:

```
items = query.offset(pagination.offset).limit(pagination.limit).all()
# Returns exactly 20 rows (configurable) instead of millions
```

17.4 Async ML Training

```
if data.run_async:
    # Returns in <100ms — training happens in background Celery worker
    task = run_training_pipeline_task.delay(...)
    return TrainResponse(status="QUEUED", task_id=task.id)
```

Always use async training in production. Synchronous training can take minutes and will timeout API gateways.

17.5 ML Batch Scoring

```
# Efficient: score N transactions in one call
results = score_batch(db, transactions)
# sklearn RandomForest uses joblib parallelism (n_jobs=-1) across all CPU
cores
```

---

18. Testing

18.1 Test Architecture

- **Location:** `tests/` directory
- **Database:** SQLite in-memory (no PostgreSQL required)
- **Client:** FastAPI `TestClient` (httpx-based)
- **Config:** `pytest.ini`

18.2 Authentication Tests (test_auth.py)

Test Case	Expected Result
test_register_success	201, role=ANALYST
test_register_duplicate_username	409 Conflict
test_register_duplicate_email	409 Conflict
test_login_success	200, access+refresh tokens
test_login_wrong_password	401 Unauthorized
test_login_inactive_user	403 Forbidden
test_refresh_valid	200, new tokens
test_refresh_with_access_token	401 Unauthorized
test_me_authenticated	200, user profile
test_me_no_token	401 Unauthorized

```python
def test_register_always_analyst_role(client):
    response = client.post("/api/v1/auth/register", json={
        "username": "test_user",
        "email": "test@example.com",
        "password": "SecurePass123!",
        "full_name": "Test User"
    })
    assert response.status_code == 201
    assert response.json()["role"] == "ANALYST"  # Always ANALYST
    assert "hashed_password" not in response.json()  # Never expose hash
```

18.3 Rules Engine Tests (test_rules_engine.py)

Test Case	Rule	Trigger Condition
test_rule_r001_amount_threshold	R001	amount > 10,000
test_rule_r002_velocity_daily	R002	> 5 txn in 24h
test_rule_r003_velocity_hourly	R003	> 3 txn in 1h
test_rule_r004_high_risk_country	R004	country = "IR"
test_rule_r005_pep_match	R005	customer.pep_flag = True
test_rule_r006_sanctions_critical	R006	customer.sanctions_flag = True
test_rule_r007_new_counterparty	R007	First-time counterparty

Test Case	Rule	Trigger Condition
test_rule_r008_weekend	R008	Saturday or Sunday
test_rule_r009_round_amount	R009	amount = 5000.00
test_rule_r010_rapid_succession	R010	2 txn within 60s
test_score_capped_at_100	All	Multiple rules fire
test_sanctions_recommends_sar	R006	SAR action

```
def test_rule_r001_amount_threshold(db, customer, account):
    txn = Transaction(amount=15000.00, ...)
    db.add(txn); db.flush()

    rules, score, alert = apply_to_transaction(db, txn, customer)
    assert "R001" in [r.rule_id for r in rules]
    assert score >= 25  # MEDIUM weight = 25

def test_score_capped_at_100(db, sanctioned_customer, account):
    # R006 (CRITICAL=90) + R001 (MEDIUM=25) = 115, capped to 100
    txn = Transaction(amount=15000.00, ...)
    db.add(txn); db.flush()
    _, score, _ = apply_to_transaction(db, txn, sanctioned_customer)
    assert score == 100
```

18.4 ML Tests (test_ml.py)

Test Case	Expected Result
test_score_without_models	409 Conflict
test_score_transaction_success	200, valid ML response
test_score_batch	200, list of scores
test_train_requires_admin	403 for ANALYST role
test_train_success	200, COMPLETED
test_list_models_empty	200, empty list
test_performance_no_feedback	200, null precision

18.5 Running Tests

```
# Run all tests
pytest tests/ -v
```

```
# With coverage report
pytest tests/ --cov=app --cov-report=html
open htmlcov/index.html

# Run specific test file
pytest tests/test_rules_engine.py -v

# Run tests matching pattern
pytest tests/ -k "test_rule" -v

# Stop on first failure
pytest tests/ -x -v
```

18.6 Test Configuration (pytest.ini)

```
[pytest]
testpaths = tests
python_files = test_*.py
python_classes = Test*
python_functions = test_*
addopts = -v --tb=short
```

---

19. Error Handling

19.1 Error Response Format

FastAPI returns structured error responses:

```
{
  "detail": "Transaction not found"
}
```

For validation errors (422):

```
{
  "detail": [
    {
      "type": "greater_than",
      "loc": ["body", "amount"],
      "msg": "Input should be greater than 0",
```

```
        "input": -500.0
      }
    ]
  }
```

19.2 HTTP Status Code Reference

Code	Meaning	When Used in KYRO
200 OK	Success	GET requests, successful ML scoring
201 Created	Resource created	POST /transactions, POST /customers
400 Bad Request	Business logic error	Account does not belong to customer
401 Unauthorized	Authentication failure	Invalid/expired JWT
403 Forbidden	Authorization failure	Insufficient role (ANALYST trying to train)
404 Not Found	Resource not found	Customer/transaction/account ID not in DB
409 Conflict	State conflict	Duplicate username, models not trained yet
422 Unprocessable Entity	Validation error	Invalid amount (negative), missing field
500 Internal Server Error	Unhandled exception	Unexpected code error

19.3 Error Catalog

AUTH-001: Invalid Credentials

- **Endpoint:** POST /auth/login
- **Status:** 401
- **Cause:** Wrong username or password
- **Message:** "Incorrect username or password"
- **Reproduction:** Login with a wrong password
- **Fix:** Provide correct credentials; reset password if forgotten

AUTH-002: Inactive User

- **Endpoint:** POST /auth/login
- **Status:** 403
- **Cause:** User account has is_active=False

- **Message:** "User is inactive"
- **Fix:** Admin must set user.is_active=True in DB

AUTH-003: Invalid Token

- **Endpoint:** Any protected endpoint
- **Status:** 401
- **Cause:** Expired JWT, wrong signature, or malformed token
- **Message:** "Could not validate credentials"
- **Fix:** Re-login to get fresh tokens

TXN-001: Customer Not Found

- **Endpoint:** POST /transactions
- **Status:** 404
- **Cause:** customer_id UUID not in customers table
- **Fix:** Create customer first via POST /customers

TXN-002: Account-Customer Mismatch

- **Endpoint:** POST /transactions
- **Status:** 400
- **Cause:** account.customer_id != customer_id in request
- **Message:** "Account does not belong to customer"
- **Fix:** Use an account that belongs to the specified customer

ML-001: Models Not Trained

- **Endpoint:** POST /ml/score-transaction
- **Status:** 409
- **Cause:** No .pkl model files exist in models/ directory
- **Message:** "Models not trained yet: No versions found for model 'risk_scorer'"
- **Fix:** POST /api/v1/ml/train (as ADMIN) with sufficient transaction data

ML-002: Insufficient Training Data

- **Endpoint:** POST /ml/train
- **Status:** 409
- **Cause:** No transactions in database to train on
- **Message:** "Insufficient training data"
- **Fix:** Ingest transactions first via POST /transactions

DB-001: Connection Failure

- **Symptom:** 500 Internal Server Error on any DB-dependent endpoint
- **Cause:** PostgreSQL container not healthy, wrong DATABASE_URL
- **Fix:** `docker compose ps` to check postgres health; verify DATABASE_URL in .env

RATE-001: Celery Not Connected

- **Symptom:** POST /ml/train with run_async=true returns no task_id
- **Cause:** Redis broker unavailable
- **Fix:** `docker compose ps` to check redis health; verify REDIS_URL in .env

19.4 Best Practices for Error Handling

1. **Never expose internal errors to clients:** Use try/except around ML operations
2. **Log errors with context:** Include transaction_id, user_id in log messages
3. **Use HTTPException, not bare exceptions:** FastAPI converts these to JSON automatically
4. **Validate at API boundary:** Pydantic catches bad inputs before business logic runs
5. **Fail fast:** Check prerequisites (customer exists, account belongs to customer) early

---

20. Business Logic

20.1 Rules Engine Logic (R001-R010)

R001: Amount Threshold

```
Rule: transaction.amount > $10,000
Severity: MEDIUM (weight = 25)
Why: US Bank Secrecy Act requires CTR filing for cash transactions > $10,000
Edge cases:
  - amount == 10000.00 → NOT triggered (strictly greater than)
  - amount == 10000.01 → triggered
```

R002: Daily Velocity

```
Rule: transactions by same customer in last 24h > 5
Severity: MEDIUM (weight = 25)
Why: Structuring — breaking large amounts into many small transactions
Calculation:
  daily_count = COUNT(txn WHERE customer_id=x AND date BETWEEN now-24h AND
```

```
now)
  if daily_count + 1 > 5: trigger
```

R003: Hourly Velocity

```
Rule: transactions by same customer in last 1h > 3
Severity: LOW (weight = 10)
Why: Rapid automated transactions may indicate account takeover or bot
```

R004: High Risk Country

```
Rule: txn.meta_country OR meta_destination_country OR meta_origin_country
      is in HIGH_RISK_COUNTRIES set
Severity: HIGH (weight = 50)
HIGH_RISK_COUNTRIES = {IR, KP, SY, MM, AF, YE, SS} + full names
Why: FATF-designated high-risk jurisdictions with AML/CFT deficiencies
```

R005: PEP Match

```
Rule: customer.pep_flag == True
Severity: HIGH (weight = 50)
Why: Politically Exposed Persons are higher risk due to corruption potential
     FATF Recommendation 12 mandates enhanced due diligence for PEPs
```

R006: Sanctions Match

```
Rule: customer.sanctions_flag == True
Severity: CRITICAL (weight = 90)
Recommended action: SAR (Suspicious Activity Report) — mandatory filing
Why: Transacting with sanctioned individuals is a federal crime
     (OFAC violations can result in unlimited civil and criminal penalties)
```

R007: New Counterparty

```
Rule: txn.meta_counterparty not in historical counterparties
Severity: LOW (weight = 10)
Why: Novel counterparties are a weak signal; combined with other flags becomes
significant
```

R008: Weekend Activity

```
Rule: txn.transaction_date.weekday() >= 5 (Saturday or Sunday)
Severity: LOW (weight = 10)
Why: Corporate business transactions rarely happen on weekends
     Weekend activity is unusual, especially for corporate customers
```

R009: Round Amount

```
Rule: txn.amount >= 1000 AND txn.amount % 1000 == 0
Severity: MEDIUM (weight = 25)
Why: Structuring detection — criminals often use round amounts
     Examples: $5,000, $10,000, $25,000 are suspicious
```

R010: Rapid Succession

```
Rule: another transaction from same customer within 60 seconds
Severity: HIGH (weight = 50)
Why: Multiple transactions within 60 seconds suggests automated fraud
     Normal human behavior doesn't produce multiple txn/minute
```

20.2 Risk Score Calculation

```
score = sum(SEVERITY_WEIGHT[r.severity] for r in triggered_rules)
score = min(100, score)

SEVERITY_WEIGHT = {"LOW": 10, "MEDIUM": 25, "HIGH": 50, "CRITICAL": 90}

Example 1: Only R001 fires
  score = 25
  No alert (below ALERT_THRESHOLD=50)

Example 2: R001 + R004 fire
  score = 25 + 50 = 75
  Alert: ENHANCED_DUE_DILIGENCE

Example 3: R006 fires (CRITICAL)
  score = 90
  Alert: SAR

Example 4: R001 + R004 + R005 fire
  score = min(100, 25+50+50) = 100
  Alert: ENHANCED_DUE_DILIGENCE (score>=75)
  But also: SAR? No — SAR requires CRITICAL rule (R006), not just high score
```

20.3 Alert Routing Logic

```
def recommended_action_for(rules, score) -> str | None:
    if any(r.severity == "CRITICAL" for r in rules):
        return "SAR"                          # Mandatory SAR filing
    if score >= 75:
        return "ENHANCED_DUE_DILIGENCE"       # EDD required
    if score >= 50:                            # ALERT_THRESHOLD
        return "REVIEW"                        # Standard analyst review
    return None                                # No alert
```

20.4 ML Alert Routing

```
# Phase 2 (ML) thresholds (different from Phase 1 rules thresholds):
LOW_MAX = 30      # combined_score <= 30: no alert, log only
MEDIUM_MAX = 70   # 30 < score <= 70: BATCH_REVIEW alert
                  # score > 70: IMMEDIATE_REVIEW alert

if risk_score <= LOW_MAX:
    return None  # No alert — only ml_scores record
elif risk_score <= MEDIUM_MAX:
    action = "BATCH_REVIEW"    # Can wait for next analyst shift
else:
    action = "IMMEDIATE_REVIEW"  # Alert immediately
```

20.5 Customer Risk Profile Update

After ML scoring all recent transactions via POST /ml/score-customer:

```
overall_risk = sum(r["risk_score"] for r in results) / len(results)
risk_level = "HIGH" if overall_risk > 70 else "MEDIUM" if overall_risk > 30
else "LOW"

customer.risk_score = round(overall_risk)
customer.risk_level = risk_level
db.commit()
```

20.6 Trend Analysis

```
midpoint = cutoff + (now - cutoff) / 2
first_half = [r for t, r in zip(txns, results) if t.date < midpoint]
second_half = [r for t, r in zip(txns, results) if t.date >= midpoint]
```

```
if avg_second > avg_first * 1.1:    # > 10% increase
    trend = "Risk trending upward over the last 30 days"
elif avg_second < avg_first * 0.9:  # > 10% decrease
    trend = "Risk trending downward over the last 30 days"
else:
    trend = "Risk stable over the last 30 days"
```

---

21. Mathematical Explanations

21.1 Z-Score (Standard Score)

Formula:

```
z = (x - mu) / sigma

where:
  x = observed value (transaction amount)
  mu = population mean (global average amount)
  sigma = population standard deviation

Example:
  Global mean amount = $5,000
  Global std dev = $4,000
  Transaction amount = $22,000

  z = (22,000 - 5,000) / 4,000
    = 17,000 / 4,000
    = 4.25

Interpretation:
  z=0 → exactly at average
  z=1 → 1 standard deviation above average (84th percentile)
  z=2 → 2 std dev above average (97.7th percentile)
  z=4.25 → 99.999th percentile — extremely unusual
```

Business meaning: A z-score of 4.25 means this transaction amount is more extreme than 99.999% of all historical transactions. This strongly suggests either a legitimate but unusual large transaction, or a suspicious one.

21.2 Percentile Calculation

Simple explanation: If your transaction amount is at the 90th percentile, it means 90% of all transactions have a lower amount.

Implementation in KYRO:

```
SELECT PERCENTILE_CONT(0.9) WITHIN GROUP (ORDER BY amount ASC)
FROM app.transactions;
-- Returns the amount below which 90% of transactions fall
```

PostgreSQL PERCENTILE_CONT: Uses interpolation to return the exact percentile value even if it falls between two data points.

21.3 Standard Deviation

Formula:

```
sigma = sqrt((1/n) * sum((xi - mu)^2 for xi in X))

where:
  n = number of samples
  xi = individual value
  mu = mean

Example (7-day window amounts):
  Amounts: [1000, 2000, 1500, 3000, 1200]
  mu = (1000+2000+1500+3000+1200) / 5 = 1740

  Deviations squared:
    (1000-1740)^2 = 547600
    (2000-1740)^2 = 67600
    (1500-1740)^2 = 57600
    (3000-1740)^2 = 1587600
    (1200-1740)^2 = 291600

  Sum = 2552000
  Variance = 2552000 / 5 = 510400
  std = sqrt(510400) = 714.4

Interpretation: The 7-day average transaction amount varies by +/- $714 around
the mean.
If a new transaction is $5,000, it's (5000-1740)/714 = 4.56 std devs above
average - very unusual.
```

21.4 Random Forest Prediction

Simple explanation: 200 decision trees each make a prediction. The final prediction is the average.

Mathematical process:

```
For a single decision tree:
  At each node, split data based on feature threshold
  Example: if amount > 10000 → right branch; else → left branch
  Repeat until leaf node (max_depth=15 or min_samples=5)
  Leaf node prediction = average risk_score in that leaf

For Random Forest:
  prediction = (1/200) * sum(tree_i.predict(X) for i in range(200))

Example with 3 trees (simplified):
  Tree 1: predicts 72.0
  Tree 2: predicts 68.0
  Tree 3: predicts 75.0
  RF prediction = (72 + 68 + 75) / 3 = 71.67

Why averaging? Reduces variance. Individual trees overfit.
Average of 200 trees is much more stable.
```

21.5 Isolation Forest Score Normalization

Raw score from sklearn:

```
decision_function(X) returns values around 0 for normal points
  > 0 = more normal
  < 0 = more anomalous

KYRO negates this (anomaly_score = -decision_function(X)):
  higher = more anomalous (intuitive direction)
```

KYRO's 0-100 normalization:

```
isolation_to_0_100(raw_score):
  = max(0.0, min(100.0, 50.0 + raw_score * 50.0))

Examples:
  raw_score = 0.0 (borderline) → 50.0
  raw_score = 0.8 (anomalous) → min(100, 50 + 40) = 90.0
  raw_score = -0.6 (normal) → max(0, 50 - 30) = 20.0
  raw_score = 1.5 (very anomalous) → min(100, 50 + 75) = 100.0
```

21.6 Weighted Ensemble Formula

Full worked example:

```
Given:
  rf_risk_score = 68.0       (RF Regressor output)
  anomaly_probability = 0.73  (RF Classifier probability of anomaly)
  isolation_score = 72.0     (Isolation Forest normalized score)

Step 1: anomaly_probability * 100 = 73.0

Step 2: Apply weights:
  0.50 * 68.0 = 34.0
  0.35 * 73.0 = 25.55
  0.15 * 72.0 = 10.8

Step 3: Sum: 34.0 + 25.55 + 10.8 = 70.35

Step 4: Clamp to [0, 100]: 70.35 (already in range)

Step 5: Anomaly flag:
  anomaly_probability (0.73) > 0.5 → True
  OR combined_score (70.35) >= 71 → False
  result: anomaly_flag = True

Step 6: Alert routing:
  70.35 > LOW_MAX (30) and <= MEDIUM_MAX (70): BATCH_REVIEW alert
  (Note: 70.35 > 70, so actually IMMEDIATE_REVIEW)
```

---

22. Interview Questions & Answers

22.1 Developer Questions

Q1: What is FastAPI and how does it differ from Flask?

A: FastAPI is a modern Python web framework built on Starlette and Pydantic. Key differences from Flask:

- FastAPI has native async/await support (asynchronous by design)
- Auto-generates Swagger/OpenAPI documentation from code
- Uses Pydantic for request/response validation (much faster than Flask's manual validation)
- Type annotations are used for routing and validation

- Flask is simpler but requires extensions for all of these features

Q2: Explain the purpose of `@lru_cache` on `get_settings()`

A: `@lru_cache(maxsize=1)` makes `get_settings()` compute its result only once per Python process. On the first call, it reads and parses the `.env` file. On subsequent calls (for every API request), it returns the cached `Settings` object without any file I/O. This is a performance optimization — reading environment files is I/O-bound and would be wasteful on every request.

Q3: Why is `pool_pre_ping=True` important in the SQLAlchemy engine?

A: Database connections can become "stale" — the TCP connection appears open in Python's pool but the database server has already dropped it (due to firewall timeout or DB restart). Without `pool_pre_ping`, the first query on a stale connection raises an error. With `pool_pre_ping=True`, SQLAlchemy sends a `SELECT 1` before each connection use to verify it's alive. If it's dead, it reconnects transparently.

Q4: Why does `audit_logs.performed_by` have no foreign key to `users`?

A: Intentional design. If a user is deleted, their audit log entries must still exist for regulatory and forensic purposes. A foreign key with `ON DELETE CASCADE` would delete audit logs when users are deleted (bad). `ON DELETE SET NULL` would work but `performed_by` would lose its value. The current design (no FK) ensures audit logs are truly immutable and survive user lifecycle changes.

Q5: Explain the difference between `db.flush()` and `db.commit()`

A:

- `db.flush()`: Sends the SQL INSERT/UPDATE to the database, but does NOT commit the transaction. The changes are visible within the current session (you can query them) but not to other sessions. Used to get generated IDs before committing.
- `db.commit()`: Permanently commits all changes in the current transaction. Makes changes visible to all connections. If the commit fails, changes are rolled back.

In the transaction ingestion flow: `db.flush()` is called after creating the transaction (to get its UUID), then the rules engine runs (using that UUID to create risk flags), then `db.commit()` at the very end commits everything atomically.

22.2 ML Engineer Questions

Q6: Why use Random Forest instead of Gradient Boosting (XGBoost/LightGBM)?

A: For this use case:

- Random Forest works well without hyperparameter tuning (our first version needs to be maintainable)
- SHAP TreeExplainer is perfectly optimized for Random Forest and produces exact (not approximate) explanations
- Random Forest training is trivially parallelizable (n_jobs=-1)
- The ML targets (rules engine scores) are noisy proxy labels — Random Forest's bagging handles this well
- XGBoost/LightGBM would likely give better raw performance but add complexity

In production v2, LightGBM could be benchmarked against this baseline.

Q7: Why `class_weight="balanced"` in the AnomalyClassifier?

A: In AML, genuine anomalies (money laundering) make up < 5% of all transactions. Without correction, a classifier can achieve 95% accuracy by always predicting "not anomalous" — completely useless.

`balanced` automatically adjusts sample weights: minority class (anomalous, 5%) gets weight = n_total / (n_classes *n_anomalous) = 1000/(2*50) = 10, meaning each anomalous sample counts as 10 normal samples during training. This forces the model to pay more attention to rare anomalies.

Q8: Explain SHAP values and why they're used for regulatory compliance

A: SHAP (SHapley Additive exPlanations) computes the contribution of each feature to a specific prediction using game theory. For a risk score of 82:

- base_value = 35 (average prediction)
- amount_zscore contributes +18 (pushes score up)
- high_risk_country contributes +12
- remaining features contribute +17

Sum: 35 + 47 = 82. SHAP values always add up to the prediction (additivity property).

For regulatory compliance: financial institutions must justify why a transaction was flagged. SHAP provides this in plain English ("Amount deviates significantly from the global baseline") rather than a black-box score.

Q9: What is Isolation Forest and how does it differ from supervised anomaly detection?

A: Isolation Forest is unsupervised — it requires no labels. It works by:

1. Randomly selecting a feature and a random split value
2. Anomalies are "isolated" quickly (few splits) because they're in sparse regions

3. Normal points require many splits because they're in dense regions

Difference from supervised (Random Forest Classifier):

- Isolation Forest: no labels needed; detects any statistical outlier
- RF Classifier: needs labeled examples; learns specific patterns from training data
- Combination: IF catches novel anomaly patterns the classifier hasn't seen during training

Q10: How does the model registry enable zero-downtime model updates?

A: The registry stores two version pointers: "active" (serving 90% of traffic) and "candidate" (10%). Both versions are `.pkl` files on disk. During a scoring request:

```
if random.random() * 100 < candidate_traffic_pct:
    return candidate_version  # 10% of requests
return active_version         # 90% of requests
```

If the candidate performs better (lower false positive rate), `promote_candidate()` is called which just updates the JSON file. No restart needed. Old model files remain on disk for rollback.

22.3 Database Questions

Q11: When would you use a BRIN index vs a B-tree index?

A: Use BRIN when:

- Data is stored in physical order correlated with the indexed column
- The column is time-series/append-only (transactions.transaction_date)
- Table is very large (millions of rows)
- Storage cost is a concern (BRIN is 100-1000x smaller than B-tree)

Use B-tree when:

- Data is random (user.email — emails are inserted in any order)
- Need exact equality lookups or arbitrary range queries
- Table size is manageable (< 10M rows)

In KYRO: `transaction_date` uses BRIN (transactions inserted in chronological order). `customer_id`, `account_id` FKs use B-tree (random UUIDs).

Q12: Explain the GIN index on risk_flags JSONB

A: GIN (Generalized Inverted Index) inverts JSONB structure — it creates an index entry for each key-value pair in the JSONB document. This enables containment queries:

```
-- Find all transactions with R006 in triggered_rules:
SELECT * FROM transactions WHERE risk_flags @> '{"triggered_rules":
["R006"]}';
```

Without GIN: full table scan on every JSONB column (sequential scan of all rows).
With GIN: direct index lookup for matching key-value pairs — much faster at scale.

Q13: Why is PostgreSQL used instead of MongoDB for this project?

A: Financial data requires:

1. **ACID transactions** — Multiple tables (transaction + risk_flags + alerts) must update atomically
2. **Foreign key integrity** — Orphaned records (transaction without customer) are prevented at DB level
3. **CHECK constraints** — amount > 0, risk_score BETWEEN 0 AND 100 enforced at DB level
4. **Complex queries** — PERCENTILE_CONT, window functions for feature engineering
5. **JSONB** — PostgreSQL offers JSON storage with the full power of relational queries

MongoDB would require application-level ACID handling, has weaker consistency guarantees, and lacks the advanced indexing types needed for efficient time-series and containment queries.

22.4 Architecture Questions

Q14: Why is Celery used for ML training instead of a simple thread or asyncio task?

A: ML training takes minutes and needs CPU-intensive work. Issues with alternatives:

- **Thread**: GIL prevents true parallelism for CPU-bound work in Python
- **asyncio task**: asyncio is for I/O-bound work; CPU-heavy code blocks the event loop
- **Celery**: Runs in a separate process (no GIL), survives API restarts, provides monitoring, retries, and scheduling via Celery Beat

Q15: Explain the `expire_on_commit=False` setting in sessionmaker

A: By default, SQLAlchemy marks all loaded ORM objects as "expired" after a commit. The next access to any attribute would trigger a fresh SELECT query. In a FastAPI dependency flow:

1. Route handler loads a transaction
2. Handler calls `db.commit()`
3. Handler returns the transaction object in the response

Without `expire_on_commit=False`: accessing `txn.id` in the response would trigger a new SQL query (even though the transaction is already committed and in scope). With `expire_on_commit=False`: objects remain usable after commit without an extra query.

22.5 DevOps Questions

Q16: How do you ensure the API waits for PostgreSQL to be ready before starting?

A: Docker Compose `depends_on` with `condition: service_healthy`:

```
api:
  depends_on:
    postgres:
      condition: service_healthy
```

The `service_healthy` condition checks the PostgreSQL healthcheck:

```
healthcheck:
  test: ["CMD-SHELL", "pg_isready -U kyro_user -d kyro_aml"]
  interval: 10s
  retries: 5
```

The API container only starts after `pg_isready` succeeds. Without this, the API would start, fail to connect to PostgreSQL (which is still initializing), and crash.

Q17: What happens if you need to add a new database column?

A: Use Alembic migrations:

```
# Generate migration
alembic revision --autogenerate -m "Add column xyz to transactions"

# Review generated migration file
# Edit if needed (Alembic may not detect all changes correctly)

# Apply migration
alembic upgrade head

# In Docker: run migration before starting API
docker exec kyro_api alembic upgrade head
```

Never modify schema by hand in production — Alembic tracks which migrations have been applied via the `alembic_version` table.

23. User Guide

23.1 Prerequisites

- Docker Desktop installed and running
- Git installed
- At least 4GB RAM available for Docker

23.2 Installation

```
# 1. Clone repository
git clone <repository-url>
cd KYRO_NEW

# 2. Copy environment template
cp .env.example .env

# 3. Generate secure SECRET_KEY (required!)
# Windows PowerShell:
[System.Security.Cryptography.RandomNumberGenerator]::GetBytes(32) | ForEach-
Object { '{0:x2}' -f $_ } | Join-String
# Copy output and set: SECRET_KEY=<output> in .env

# 4. Start all services
docker compose up -d

# 5. Wait ~30 seconds for services to initialize
docker compose ps
# All services should show "Up" or "Up (healthy)"
```

23.3 First-Time Setup

```
# Step 1: Register an ADMIN user (via Swagger UI)
# Open http://localhost:8000/docs
# POST /api/v1/auth/register:
{
  "username": "admin_user",
  "email": "admin@yourcompany.com",
  "password": "SecurePassword123!",
  "full_name": "System Administrator"
}

# Note: Self-registered users get ANALYST role.
```

```
# To upgrade to ADMIN, connect to DB and update:
docker exec -it kyro_postgres psql -U kyro_user -d kyro_aml   -c "UPDATE
app.users SET role='ADMIN' WHERE username='admin_user';"

# Step 2: Login to get access token
# POST /api/v1/auth/login (Form data, not JSON)
# username=admin_user, password=SecurePassword123!
# Copy access_token from response

# Step 3: Click "Authorize" in Swagger UI
# Enter: Bearer <access_token>

# Step 4: Create test customers and transactions
# POST /api/v1/customers, then POST /api/v1/transactions

# Step 5: Train ML models
# POST /api/v1/ml/train with {"run_async": false}
# Wait for {"status": "COMPLETED"} response

# Step 6: Score a transaction
# POST /api/v1/ml/score-transaction with {"transaction_id": "<txn-id>"}
```

23.4 How to Run the Pipeline

```
# The pipeline runs automatically on container startup
# To run manually:
docker compose run --rm pipeline python -m pipeline.run_pipeline

# Check pipeline logs:
docker logs kyro_pipeline
```

23.5 Monitoring Services

```
# View all service logs
docker compose logs

# Follow a specific service
docker compose logs -f api
docker compose logs -f celery_worker

# Check service health
docker compose ps

# Database UI (pgAdmin)
```

```
# Open: http://localhost:5050
# Login: admin@kyro.com / admin123
# Add server: host=postgres, port=5432, db=kyro_aml, user=kyro_user
```

23.6 Common Operations

```
# Restart a specific service
docker compose restart api

# Rebuild after code changes
docker compose up -d --build api

# Stop all services
docker compose down

# Stop and remove volumes (WARNING: deletes all data)
docker compose down -v

# View resource usage
docker stats

# Execute command in container
docker exec -it kyro_api python -c "from app.config import get_settings;
print(get_settings().database_url)"
```

23.7 Troubleshooting Guide

Issue	Symptoms	Solution
API won't start	Container exits immediately	Check `docker logs kyro_api` for error; verify DATABASE_URL
409 on ML score	"Models not trained yet"	POST /api/v1/ml/train first
401 on all requests	"Could not validate credentials"	Re-login to get fresh access token
DB connection error	500 on any DB endpoint	`docker compose ps` — postgres must be healthy
Celery tasks not running	Training stays QUEUED	`docker compose ps` — celery_worker must be running
Model not found	409 on score endpoint	Check models/ directory for .pkl files
pgAdmin can't connect	Connection refused	Use host=postgres (not localhost) in pgAdmin

24. Developer Guide

24.1 Development Environment Setup

```
# Option 1: Full Docker (recommended)
docker compose up -d

# Option 2: Local Python (for faster iteration)
python -m venv venv
venv\Scripts□ctivate  # Windows
pip install -r requirements-api.txt -r requirements-ml.txt

# Start only infrastructure services
docker compose up -d postgres redis

# Run API locally
uvicorn app.main:app --reload --port 8000

# Run Celery worker locally
celery -A app.tasks.celery_app worker --loglevel=info
```

24.2 Coding Standards

Standard	Rule
Python version	3.11+ (use type hints everywhere)
Code formatter	Black (`black app/ tests/`)
Import sorting	isort (`isort app/ tests/`)
Type checking	mypy (`mypy app/`)
Docstrings	Module-level docstring required for all files
String formatting	f-strings (no % or .format())
Line length	120 characters maximum
Variable naming	snake_case for all variables and functions

24.3 How to Add a New API Endpoint

Example: Adding GET /api/v1/transactions/{id}/ml-scores

1. **Create schema** in `app/schemas/transaction.py` :

```
class MLScoreOut(BaseModel):
    id: uuid.UUID
    combined_score: float
    anomaly_flag: bool
    explanation: dict | None
    created_at: datetime

    model_config = ConfigDict(from_attributes=True)
```

2. **Add route** in `app/routers/transactions.py`:

```
from app.models.ml_score import MLScore

@router.get("/{transaction_id}/ml-scores", response_model=list[MLScoreOut])
def get_ml_scores(transaction_id: uuid.UUID,
                  db: Session = Depends(get_db)) -> list[MLScore]:
    _get_transaction_or_404(db, transaction_id)
    return db.query(MLScore).filter(
        MLScore.transaction_id == transaction_id
    ).order_by(MLScore.created_at.desc()).all()
```

3. **Write tests** in `tests/test_transactions.py`:

```
def test_get_ml_scores_success(client, auth_headers,
transaction_with_ml_score):
    response = client.get(
        f"/api/v1/transactions/{transaction_with_ml_score.id}/ml-scores",
        headers=auth_headers
    )
    assert response.status_code == 200
    assert len(response.json()) >= 1
```

4. **Run tests:**

```
pytest tests/test_transactions.py::test_get_ml_scores_success -v
```

24.4 How to Add a New ML Model

1. **Create model class** in `app/ml/models/new_model.py`:

```
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.preprocessing import StandardScaler
```

```python
import numpy as np, pandas as pd

class NewModel:
    def __init__(self):
        self.model = GradientBoostingRegressor(n_estimators=100)
        self.scaler = StandardScaler()
        self.feature_names = None

    def train(self, X: pd.DataFrame, y: pd.Series) -> None:
        self.feature_names = list(X.columns)
        X_scaled = self.scaler.fit_transform(X)
        self.model.fit(X_scaled, y)

    def predict(self, X: pd.DataFrame) -> np.ndarray:
        X_scaled = self.scaler.transform(X[self.feature_names])
        return np.clip(self.model.predict(X_scaled), 0, 100)
```

2. **Add training** in `app/ml/training/trainer.py` — instantiate and fit the model.
3. **Register model name** in `app/ml/training/pipeline.py`:

```python
MODEL_NAMES = ("risk_scorer", "anomaly_classifier", "isolation_detector",
"new_model")
```

4. **Integrate score** in `app/ml/scoring/real_time_scorer.py` — load, predict, include in ensemble.
5. **Update weights:**

```python
COMBINE_WEIGHTS = {
    "risk_scorer": 0.40,         # Adjusted
    "anomaly_classifier": 0.30, # Adjusted
    "isolation_detector": 0.15,
    "new_model": 0.15,           # New model
}
```

24.5 How to Add a New Business Rule

1. **Define rule constant** in `app/services/rules_engine.py`:

```python
NEW_RULE_THRESHOLD = 50  # Example: some configurable value
```

2. **Add rule to evaluate() function:**

```
# R011 — New Rule
if some_condition:
    triggered.append(TriggeredRule(
        "R011",
        "Rule Name",
        f"Description: {details}",
        "MEDIUM"  # LOW / MEDIUM / HIGH / CRITICAL
    ))
```

3. **Write test:**

```
def test_rule_r011(db, customer, account):
    txn = Transaction(amount=..., ...)  # Setup triggering condition
    rules, score, alert = apply_to_transaction(db, txn, customer)
    assert "R011" in [r.rule_id for r in rules]
```

24.6 Naming Conventions

Entity	Convention	Example
API endpoints	kebab-case nouns	/score-transaction, /score-batch
Python functions	snake_case verbs	score_transaction(), apply_to_transaction()
Python classes	PascalCase	RiskScorerModel, AlertRouter
Database tables	snake_case plural	ml_scores, audit_logs, transaction_risk_flags
Database columns	snake_case	risk_score, transaction_date, customer_id
Environment variables	SCREAMING_SNAKE_CASE	SECRET_KEY, DATABASE_URL
Redis cache keys	colon-delimited	ml:global_amount_stats
Celery task names	dot-delimited module path	app.tasks.etl_tasks.run_daily_etl_pipeline

24.7 Debugging Tips

```
# Interactive Python shell in container
docker exec -it kyro_api python
```

```
# Database REPL
docker exec -it kyro_postgres psql -U kyro_user -d kyro_aml

# Check Redis keys
docker exec -it kyro_redis redis-cli keys "*"

# Check Celery task results
docker exec -it kyro_redis redis-cli type celery-task-meta-<task-id>

# Inspect ML model features
docker exec -it kyro_api python -c "
from app.ml.registry.model_registry import ModelRegistry
r = ModelRegistry()
m = r.load_model('risk_scorer')
print(m['model'].feature_names)
print(m['metrics'])
"
```

---

25. Appendix

25.1 Glossary

Term	Definition
AML	Anti-Money Laundering — regulatory framework to prevent financial crime
SAR	Suspicious Activity Report — mandatory filing when money laundering is suspected
CTR	Currency Transaction Report — required for cash transactions > $10,000 (US)
PEP	Politically Exposed Person — heightened AML risk due to corruption potential
KYC	Know Your Customer — process of verifying customer identity
FATF	Financial Action Task Force — intergovernmental AML standards body
OFAC	Office of Foreign Assets Control — US sanctions authority
EDD	Enhanced Due Diligence — additional checks for high-risk customers
STR	Suspicious Transaction Report — same as SAR in some jurisdictions
JWT	JSON Web Token — compact, URL-safe token format for authentication
RBAC	Role-Based Access Control — permissions based on user role
ORM	Object-Relational Mapper — maps Python classes to database tables
SHAP	SHapley Additive exPlanations — model explainability using game theory
RF	Random Forest — ensemble of decision trees

Term	Definition
ETL	Extract, Transform, Load — data pipeline process
BRIN	Block Range INdex — compact PostgreSQL index for time-series data
GIN	Generalized Inverted Index — PostgreSQL index for JSONB/arrays
ACID	Atomicity, Consistency, Isolation, Durability — database transaction properties
CORS	Cross-Origin Resource Sharing — browser security mechanism for API access
bcrypt	Adaptive password hashing algorithm

25.2 Abbreviations

Abbreviation	Full Form
API	Application Programming Interface
DB	Database
FK	Foreign Key
PK	Primary Key
UUID	Universally Unique Identifier
JSONB	JSON Binary (PostgreSQL's efficient JSON storage format)
ML	Machine Learning
RF	Random Forest
MSE	Mean Squared Error
MAE	Mean Absolute Error
AUC	Area Under the Curve (ROC AUC)
ROC	Receiver Operating Characteristic
TTL	Time To Live (cache expiry)
CI/CD	Continuous Integration/Continuous Deployment
OWASP	Open Web Application Security Project
TLS	Transport Layer Security (HTTPS)
DDL	Data Definition Language (CREATE, ALTER, DROP)
DML	Data Manipulation Language (SELECT, INSERT, UPDATE, DELETE)

25.3 Environment Variables Reference

Variable	Default	Description
APP_NAME	KYRO Risk Assessment	Application display name
DEBUG	False	Enable debug mode
LOG_LEVEL	INFO	Logging verbosity
DATABASE_URL	postgresql+psycopg://...	Full database connection string
REDIS_URL	redis://localhost:6380/0	Redis connection string
SECRET_KEY	change-me	JWT signing key (MUST change in production)
JWT_ALGORITHM	HS256	JWT signing algorithm
ACCESS_TOKEN_EXPIRE_MINUTES	30	Access token lifetime
REFRESH_TOKEN_EXPIRE_DAYS	7	Refresh token lifetime
MODEL_REGISTRY_PATH	./models	Path to ML model storage
RETRAIN_THRESHOLD	1000	New transactions needed to trigger retraining
PERFORMANCE_THRESHOLD	0.85	Minimum acceptable precision before retraining
TRAINING_DATA_DAYS	365	Days of history to use for training
SHAP_EXPLANATION_TOP_K	5	Number of top features to include in explanation
API_HOST_PORT	8000	Host port for FastAPI service
REDIS_HOST_PORT	6380	Host port for Redis
DB_HOST	localhost	PostgreSQL host
DB_PORT	5432	PostgreSQL port
DB_NAME	kyro_aml	Database name
DB_USER	kyro_user	Database username
DB_PASSWORD	kyro_pass	Database password (CHANGE IN PRODUCTION)

25.4 API Endpoints Quick Reference

Method	Endpoint	Auth	Description
GET	/api/v1/health	None	Health check
POST	/api/v1/auth/register	None	Register user
POST	/api/v1/auth/login	None	Login, get tokens
POST	/api/v1/auth/logout	Bearer	Logout
POST	/api/v1/auth/refresh	Bearer	Refresh access token
GET	/api/v1/auth/me	Bearer	Current user profile
POST	/api/v1/customers	Bearer	Create customer
GET	/api/v1/customers	Bearer	List customers
GET	/api/v1/customers/{id}	Bearer	Get customer
PATCH	/api/v1/customers/{id}	Bearer	Update customer
POST	/api/v1/accounts	Bearer	Create account
GET	/api/v1/accounts	Bearer	List accounts
POST	/api/v1/transactions	Bearer	Ingest transaction
POST	/api/v1/transactions/batch	Bearer	Batch ingest
GET	/api/v1/transactions	Bearer	List transactions
GET	/api/v1/transactions/{id}	Bearer	Get transaction
GET	/api/v1/transactions/{id}/risk	Bearer	Get risk assessment
GET	/api/v1/transactions/{id}/flags	Bearer	Get risk flags
GET	/api/v1/alerts	Bearer	List alerts
PATCH	/api/v1/alerts/{id}	Bearer	Update alert
GET	/api/v1/kyc/reviews	Bearer	List KYC reviews
POST	/api/v1/kyc/reviews	Bearer	Create KYC review
GET	/api/v1/kyc/screenings	Bearer	List screenings
POST	/api/v1/ml/score-transaction	Bearer	ML score transaction
POST	/api/v1/ml/score-batch	Bearer	ML batch score
POST	/api/v1/ml/score-customer/{id}	Bearer	Score customer
POST	/api/v1/ml/train	ADMIN	Train ML models
GET	/api/v1/ml/models	Bearer	List model versions
GET	/api/v1/ml/performance	Bearer	Model performance

25.5 Rules Engine Quick Reference

Rule ID	Name	Severity	Weight	Trigger
R001	Amount Threshold	MEDIUM	25	amount > $10,000
R002	Velocity Daily	MEDIUM	25	> 5 txn/day
R003	Velocity Hourly	LOW	10	> 3 txn/hour
R004	High Risk Country	HIGH	50	FATF-listed country
R005	PEP Match	HIGH	50	pep_flag = True
R006	Sanctions Match	CRITICAL	90	sanctions_flag = True
R007	New Counterparty	LOW	10	First-time counterparty
R008	Weekend Activity	LOW	10	Saturday or Sunday
R009	Round Amount	MEDIUM	25	amount % 1000 = 0
R010	Rapid Succession	HIGH	50	< 60s between transactions

25.6 Version History

Version	Date	Changes
0.1.0	2026-07-30	Initial Phase 1+2 release: Rules engine + ML ensemble

25.7 References and Useful Links

Resource	URL
FastAPI Documentation	https://fastapi.tiangolo.com
SQLAlchemy 2.0 Docs	https://docs.sqlalchemy.org/en/20/
Pydantic v2 Docs	https://docs.pydantic.dev/latest/
Celery Documentation	https://docs.celeryq.dev
SHAP Documentation	https://shap.readthedocs.io
scikit-learn Docs	https://scikit-learn.org/stable/
PostgreSQL 16 Docs	https://www.postgresql.org/docs/16/
Redis Documentation	https://redis.io/docs
FATF Recommendations	https://www.fatf-gafi.org/en/topics/fatf-recommendations.html
OWASP Top 10	https://owasp.org/www-project-top-ten/
JWT RFC 7519	https://tools.ietf.org/html/rfc7519

Resource	URL
Swagger UI	http://localhost:8000/docs (when running locally)
pgAdmin	http://localhost:5050 (when running locally)

End of KYRO Enterprise Documentation v1.0.0
Generated: 2026-07-30 | Covers: Phase 1 (Rules Engine) + Phase 2 (ML Engine)
This document is intended for internal developer use only.